

Diseño e implementación de una plataforma experimental de instrumentación aviónica

Documento:

Apendices y anexos

Autor:

Arnau Mora Perez

Director/a:

Juan Mon González

Javier Gago Barrio

Titulación:

Grado en Ingeniería en Vehículos Aeroespaciales

Convocatoria:

Primavera, 2026

TRABAJO FIN DE ESTUDIOS

Intentionally left blank

Índice general

1. ANEXO I. Código Arduino	1
2. ANEXO II. Código Processing	51
3. ANEXO III. Figuras de apoyo	88

Índice de figuras

Índice de tablas

Capítulo 1

ANEXO I. Código Arduino

```
1  // FlightInstruments.ino - Sketch principal (transmisor)
2  //
3  // TFG: Diseno e implementacion de una plataforma experimental de
4  //      instrumentacion avionica.
5
6  #include <Wire.h>
7  #include "Config.h"
8  #include "SensorData.h"
9
10 // Estructuras de datos
11 RawIMU      rawIMU;
12 FlightData  flightData;
13 BaroData    baroData;
14 GPSData     gpsData;
15 RadarData   radarData;
16 ToFData     tofData;
17
18 // Timers para scheduling cooperativo. Cada subsistema corre a su
19 // frecuencia natural sin bloquear a los demas. El loop principal
20 // solo dispara la tarea que toca segun el reloj.
21 unsigned long lastSensorRead    = 0;
22 unsigned long lastBaroRead      = 0;
23 unsigned long lastSerialPrint   = 0;
24 unsigned long lastTelemetrySend = 0;
25
26 bool radioAvailable = false;
27
28 // Contadores de transmision, definidos en Telemetry.ino
29 extern uint32_t txCount;
```

```
30 extern uint32_t txFailCount;
31
32 void setup() {
33     Serial.begin(115200);
34     delay(2000);
35
36     Serial.println("  Flight Instruments");
37     Serial.println("  ESP32-S3 + MPU-9250 + BMP280 + VL53L1X");
38     Serial.println("  + NEO-6M + LD2410C + NRF24L01+PA+LNA");
39     Serial.println("  Filtro: Mahony MARG (9 ejes)");
40
41     neopixelWrite(PIN_LED_RGB, 30, 0, 0);  // rojo = booting
42
43     // XSHUT del ToF en alto para que el sensor arranque
44     pinMode(TOF_SHUT_PIN, OUTPUT);
45     digitalWrite(TOF_SHUT_PIN, HIGH);
46     delay(10);
47
48     // Interrupciones (no usadas, solo se dejan los pines configurados)
49     pinMode(TOF_IRQ_PIN, INPUT);
50     pinMode(RADAR_IRQ_PIN, INPUT);
51     pinMode(MPU_IRQ_PIN, INPUT);
52
53     // Bus I2C compartido
54     Wire.begin(PIN_SDA, PIN_SCL);
55     Wire.setClock(I2C_CLOCK);
56     Serial.println("I2C OK (400 kHz)");
57
58     // IMU. Es el unico sensor critico: si falla se bloquea el arranque
59     if (!initIMU()) {
60         Serial.println("FATAL: fallo al inicializar el IMU");
61         while (true) { // parpadeo rojo perpetuo
62             neopixelWrite(PIN_LED_RGB, 30, 0, 0); delay(300);
63             neopixelWrite(PIN_LED_RGB, 0, 0, 0); delay(300);
64         }
65     }
66
67     // Calibracion del giroscopio
68     calibrateGyro();
69
70     initFusion(flightData);
71
72     // Semilla del cuaternion con una muestra accel+mag para que el
73     // filtro Mahony arranque mas cerca de la orientacion real
```

```
74  delay(20);
75  readIMU(rawIMU);
76  seedQuaternionFromIMU(rawIMU);
77
78  // Sensores no criticos: si alguno falla el sistema sigue, solo
79  // queda marcado con ok=false y la palabra ARINC correspondiente
80  // se transmite con SSM=NO_DATA
81  baroData.ok = initBarometer();
82  if (!baroData.ok) Serial.println("AVISO: barometro offline, sin altitud
    barometrica");
83
84  tofData.ok = initToF();
85  if (!tofData.ok) Serial.println("AVISO: ToF offline, sin distancia al suelo")
    ;
86
87  initGPS(); // el flag ok se activa en el primer readGPS() valido
88  radarData.ok = initRadar();
89
90  radioAvailable = initTelemetry();
91  if (!radioAvailable) Serial.println("AVISO: radio offline, sigue sin
    telemetria");
92
93  #ifndef CALIBRATE_MAG
94    runMagCalOperational(); // solo la LLAMADA; captura y luego halt
95  #endif
96
97  neopixelWrite(PIN_LED_RGB, 0, 30, 0); // verde = listo
98  Serial.println("\nSistema listo. Streaming...\n");
99 }
100
101 void loop() {
102   unsigned long nowUs = micros();
103   unsigned long nowMs = millis();
104
105   // Alimentamos buffers de UART cada iteracion (no bloqueante).
106   // Si se descuida aunque sea un ciclo, la UART se satura y se pierden frames.
107   feedGPS();
108   feedRadar();
109
110   // IMU + fusion a 100 Hz
111   if (nowUs - lastSensorRead >= SENSOR_INTERVAL_US) {
112     lastSensorRead = nowUs;
113     readIMU(rawIMU);
114     updateFusion(rawIMU, flightData);
```



```
115     }
116
117     // Barometro a 10 Hz
118     if (baroData.ok && (nowMs - lastBaroRead >= BARO_INTERVAL_MS)) {
119         lastBaroRead = nowMs;
120         readBarometer(baroData);
121     }
122
123     // ToF (no bloqueante, solo actualiza si hay muestra nueva)
124     readToF(tofData);
125
126     // GPS
127     readGPS(gpsData);
128
129     // Radar (recibe la velocidad GPS para filtrar detecciones estaticas o en
130     movimiento)
131     float platformSpeed = gpsData.fix ? gpsData.speed_kmh : 0.0f;
132     readRadar(radarData, platformSpeed);
133
134     // Telemetria a 20 Hz
135     if (radioAvailable && (nowMs - lastTelemetrySend >= TELEMETRY_INTERVAL_MS)) {
136         lastTelemetrySend = nowMs;
137         sendTelemetry(flightData, baroData, gpsData, radarData, tofData);
138     }
139
140     // Debug por serial a 10 Hz. Solo se compila si DEBUG_SERIAL esta
141     // definido (ver Config.h) ya que el transmisor normalmente no esta
142     // conectado por USB, asi se ahorra computacion.
143     #ifdef DEBUG_SERIAL
144     if (nowMs - lastSerialPrint >= SERIAL_PRINT_INTERVAL) {
145         lastSerialPrint = nowMs;
146
147         // Magnitud del campo magnetico para diagnostico
148         float magMag = sqrtf(rawIMU.mx * rawIMU.mx
149                               + rawIMU.my * rawIMU.my
150                               + rawIMU.mz * rawIMU.mz);
151
152         float lossPct = (txCount > 0) ? (txFailCount * 100.0f / txCount) : 0.0f;
153         Serial.printf(
154             "P: %.1f R: %.1f H: %.1f |Mag: %.1fuT| Alt: %.1f VS: %.1f | Sat: %d %s"
155             " | Rdr: %ucm ToF: %umm | TX: %lu L: %.1f%%\n",
156             flightData.pitch, flightData.roll, flightData.heading, magMag,
157             baroData.altitude_m, baroData.vspeed_mps,
```

```
158     gpsData.satellites, gpsData.fix ? "Fix" : "NoFix",
159     radarData.bestDist_cm, tofData.distance_mm,
160     txCount, lossPct
161 );
162 }
163 #endif
164 }
165
166
167
168 // Calibracion del magnetometro
169 // Calibracion hard/soft iron con el sistema completo en operacion.
170
171 #ifndef CALIBRATE_MAG
172 extern float magCalX, magCalY, magCalZ; // ASA de fabrica, definidos en IMU.
173     ino
174
175 void runMagCalOperational() {
176     Serial.println("\nCAL MAG: Esperando fix GPS con el sistema en operacion...")
177     ;
178     unsigned long tStart = millis(), tWarmTx = 0, tWarmPrint = 0;
179
180     // Bucle que no termina hasta que el GPS obtenga un Fix válido
181     while (!gpsData.fix) {
182         // Reproduce el loop normal para mantener el sistema vivo
183         feedGPS(); feedRadar();
184         readIMU(rawIMU);
185         updateFusion(rawIMU, flightData);
186         readBarometer(baroData);
187         readToF(tofData);
188         readGPS(gpsData);
189         readRadar(radarData, gpsData.fix ? gpsData.speed_kmh : 0.0f);
190
191         // Comprobamos radioAvailable antes de transmitir
192         if (radioAvailable && (millis() - tWarmTx >= TELEMETRY_INTERVAL_MS)) {
193             tWarmTx = millis();
194             sendTelemetry(flightData, baroData, gpsData, radarData, tofData);
195         }
196
197         // Imprimir el estado cada segundo para ver el progreso
198         if (millis() - tWarmPrint > 1000) {
199             tWarmPrint = millis();
200             Serial.print("WARMUP: Esperando GPS... Tiempo transcurrido: ");
201             Serial.print((millis() - tStart) / 1000UL);
```

```

200     Serial.print("s  GPS: NoFix");
201     Serial.print("  Sat: ");
202     Serial.println(gpsData.satellites);
203 }
204 }
205
206 Serial.println("\nGPS Fix conseguido. Dando 5 segundos de estabilizacion
    final...");
207 delay(5000);
208
209 Serial.println("\nCAL MAG: rotar LENTO en los 3 ejes durante 30 s");
210 for (int i = 5; i > 0; i--) { Serial.print(i); Serial.print("... "); delay
    (1000); }
211 Serial.println("\n>>> ROTAR AHORA <<<\n");
212
213 float minX=1e6f, minY=1e6f, minZ=1e6f;
214 float maxX=-1e6f, maxY=-1e6f, maxZ=-1e6f;
215 unsigned long t0 = millis(), tTx = 0, tPrint = 0;
216 uint32_t n = 0;
217
218 while (millis() - t0 < 30000UL) {
219     // Reproduce el loop normal: radio TX a 20 Hz + todos los sensores activos
220     // Asi la calibracion se hace con los sensores en funcionamiento
221     feedGPS(); feedRadar();
222     readIMU(rawIMU);
223     updateFusion(rawIMU, flightData);
224     readBarometer(barodata);
225     readToF(tofData);
226     readGPS(gpsData);
227     readRadar(radarData, gpsData.fix ? gpsData.speed_kmh : 0.0f);
228
229     // Comprobamos radioAvailable antes de transmitir
230     if (radioAvailable && (millis() - tTx >= TELEMETRY_INTERVAL_MS)) {
231         tTx = millis();
232         sendTelemetry(flightData, barodata, gpsData, radarData, tofData);
233     }
234
235     // Mag SOLO con ASA (sin offset/scale)
236     Wire.beginTransmission(AK8963_ADDR); Wire.write(0x03); Wire.endTransmission
        (false);
237     Wire.requestFrom(AK8963_ADDR, (byte)7);
238     if (Wire.available() >= 7) {
239         int16_t xr = Wire.read() | (Wire.read() << 8);
240         int16_t yr = Wire.read() | (Wire.read() << 8);

```

```

241     int16_t zr = Wire.read() | (Wire.read() << 8);
242     (void)Wire.read(); // ST2, obligatorio leerlo
243     float mx = xr * MAG_SCALE * magCalX;
244     float my = yr * MAG_SCALE * magCalY;
245     float mz = zr * MAG_SCALE * magCalZ;
246     if (mx<minX) minX=mx; if (mx>maxX) maxX=mx;
247     if (my<minY) minY=my; if (my>maxY) maxY=my;
248     if (mz<minZ) minZ=mz; if (mz>maxZ) maxZ=mz;
249     n++;
250 }
251
252 if (millis() - tPrint > 1000) {
253     tPrint = millis();
254     Serial.print("CAL t-"); Serial.print((30000UL - (millis()-t0)) / 1000UL);
255     Serial.print("s N="); Serial.println(n);
256 }
257 }
258
259 // Offset = centro del elipsoide. escala = correccion soft iron por eje
260 float offX=(maxX+minX)*0.5f, offY=(maxY+minY)*0.5f, offZ=(maxZ+minZ)*0.5f;
261 float rX=(maxX-minX)*0.5f, rY=(maxY-minY)*0.5f, rZ=(maxZ-minZ)*0.5f;
262 float avg=(rX+rY+rZ)/3.0f;
263 float scX=(rX>0.1f)?avg/rX:1.0f, scY=(rY>0.1f)?avg/rY:1.0f, scZ=(rZ>0.1f)?avg
    /rZ:1.0f;
264
265 Serial.println("\nCALIBRACION COMPLETADA");
266 Serial.print("Muestras: "); Serial.println(n);
267 Serial.print("Magnitud media: "); Serial.print(avg,1);
268 Serial.println(" uT (esperado ~46 uT, WMM2025 Terrassa)");
269 Serial.println("\nPEGAR EN Config.h (sustituir las 6 lineas existentes):\n");
270 Serial.print("#define MAG_OFFSET_X    "); Serial.print(offX,4); Serial.
    println("f");
271 Serial.print("#define MAG_OFFSET_Y    "); Serial.print(offY,4); Serial.
    println("f");
272 Serial.print("#define MAG_OFFSET_Z    "); Serial.print(offZ,4); Serial.
    println("f");
273 Serial.print("#define MAG_SCALE_X     "); Serial.print(scX,4); Serial.
    println("f");
274 Serial.print("#define MAG_SCALE_Y     "); Serial.print(scY,4); Serial.
    println("f");
275 Serial.print("#define MAG_SCALE_Z     "); Serial.print(scZ,4); Serial.
    println("f");
276
277 while (true) { // halt: no seguir al loop normal. Reflashear sin

```

```
        CALIBRATE_MAG.  
278     neopixelWrite(PIN_LED_RGB, 0, 30, 0); delay(500);  
279     neopixelWrite(PIN_LED_RGB, 0, 0, 0); delay(500);  
280 }  
281 }  
282 #endif
```

Código 1.1: Sketch principal: setup y loop

```
1 // Config.h - constantes de hardware y parametros de ajuste  
2  
3 #ifndef CONFIG_H  
4 #define CONFIG_H  
5  
6 // DEBUG  
7 // #define DEBUG_SERIAL  
8  
9 // MAG Calibration  
10 // #define CALIBRATE_MAG  
11  
12  
13 // Definicion Pines  
14  
15 // I2C compartido (MPU-9250 + BMP280 + VL53L1X)  
16 #define PIN_SDA      8  
17 #define PIN_SCL      9  
18  
19 // LED RGB onboard  
20 #define PIN_LED_RGB  48  
21  
22 // NRF24L01+PA+LNA (SPI)  
23 #define NRF_SCK      39  
24 #define NRF_MISO     40  
25 #define NRF_MOSI     41  
26 #define NRF_CE       2  
27 #define NRF_CSN     42  
28 #define NRF_IRQ     38  
29  
30 // GPS NEO-6M (UART1)  
31 #define GPS_RX_PIN   21 // ESP32 RX <- GPS TX  
32 #define GPS_TX_PIN   47 // ESP32 TX -> GPS RX  
33 #define GPS_BAUD     9600
```

```

34
35 // HLK-LD2410C radar (UART2)
36 #define RADAR_RX_PIN 17
37 #define RADAR_TX_PIN 18
38 #define RADAR_BAUD 256000
39
40 // Interrupciones / control (no usadas en el código, pero asignadas)
41 #define TOF_SHUT_PIN 11 // XSHUT del TOF400C
42 #define TOF_IRQ_PIN 10 // INT del TOF400C
43 #define RADAR_IRQ_PIN 16
44 #define MPU_IRQ_PIN 4
45
46 // Direcciones I2C
47 #define MPU9250_ADDR 0x68 // ADO a GND
48 #define AK8963_ADDR 0x0C // magnetometro en el MPU-9250
49 #define BMP280_ADDR 0x76 // SDO a GND
50 #define VL53L1X_ADDR 0x29 // direccion por defecto del TOF400C
51 #define I2C_CLOCK 400000 // 400 kHz (Fast Mode)
52
53 // Rangos del MPU-9250
54 // Factores sacados del datasheet PS-MPU-9250A-01 Rev 1.1
55 #define ACCEL_SCALE 8192.0 // LSB/g a +/-4g
56 #define GYRO_SCALE 65.5 // LSB/(deg/s) a +/-500deg/s
57 #define MAG_SCALE 0.15 // uT/LSB en 16 bits
58
59 // BMP280
60 #define SEA_LEVEL_HPA 1013.25f // presion de referencia ISA
61 #define VSPEED_ALPHA 0.2f // suavizado EMA de la velocidad vertical
62
63 // VL53L1X / TOF400C
64 #define TOF_TIMING_BUDGET_US 50000 // 50 ms modo Long permite hasta 4 m
65 #define TOF_CONTINUOUS_MS 50 // 20 Hz
66 #define TOF_DEAD_ZONE_MM 40 // por debajo de aqui la medida no es
    fiable
67
68 // HLK-LD2410C
69
70 // Resolucion del gate.
71 #define RADAR_GATE_WIDTH_CM 20
72
73 // Thresholds por gate (downward-facing)
74 #define RADAR_SENS_GATE_0 80 // 0.00 - 0.20 m
75 #define RADAR_SENS_GATE_1 70 // 0.20 - 0.40 m
76 #define RADAR_SENS_GATE_2 60 // 0.40 - 0.60 m

```

```

77 #define RADAR_SENS_GATE_3      50    // 0.60 - 0.80 m
78 #define RADAR_SENS_GATE_FAR    35    // 0.80 - 1.80 m (gates 4-8)
79
80 // Peak-finder (modo enhanced)
81 #define RADAR_NOISE_FLOOR      40
82
83 // Suavizado EMA de la distancia derivada por gate
84 #define RADAR_DIST_ALPHA      0.40f
85
86 #define RADAR_HOVER_SPEED_KMH  2.0f  // por debajo de esta velocidad se
      considera hover
87 #define RADAR_ALERT_ENERGY_MIN 30    // energia minima para disparar alerta
88
89 // Zona de proximidad en funcion de la distancia
90 #define RADAR_ZONE_NONE        0
91 #define RADAR_ZONE_DANGER      1    // 0.00 - 0.60 m (gates 0-2)
92 #define RADAR_ZONE_CAUTION     2    // 0.60 - 1.20 m (gates 3-5)
93 #define RADAR_ZONE_CLEAR       3    // 1.20 - 1.80 m (gates 6-8)
94
95 // Tasas de muestreo
96 #define SENSOR_RATE_HZ         100    // IMU + fusion
97 #define SENSOR_INTERVAL_US     (1000000 / SENSOR_RATE_HZ)
98
99 #define BARO_RATE_HZ           10
100 #define BARO_INTERVAL_MS       (1000 / BARO_RATE_HZ)
101
102 #define RADAR_PRINT_HZ         2
103 #define RADAR_PRINT_INTERVAL   (1000 / RADAR_PRINT_HZ)
104
105 #define SERIAL_PRINT_HZ        10
106 #define SERIAL_PRINT_INTERVAL  (1000 / SERIAL_PRINT_HZ)
107
108 // Calibracion del giroscopio
109 #define CALIBRATION_SAMPLES     500
110 #define CALIBRATION_VARIANCE_THRESHOLD 2.0
111
112 // Mahony
113 #define MAHONY_KP               3.0f
114 #define MAHONY_KI               1.0f
115
116
117 // CALIBRACION DEL MAGNETOMETRO (Hard iron / Soft iron)
118
119 // Hard iron offsets (uT)

```

```

120 #define MAG_OFFSET_X      10.6264f
121 #define MAG_OFFSET_Y      -2.6479f
122 #define MAG_OFFSET_Z      -30.1492f
123
124 // Soft iron scales (adimensional)
125 #define MAG_SCALE_X        1.0157f
126 #define MAG_SCALE_Y        0.9977f
127 #define MAG_SCALE_Z        0.9884f
128
129 // CORRECCIONES DE HEADING
130 //
131 // Declinacion magnetica
132 // Diferencia entre el norte magnetico (lo que mide el sensor) y el
133 // norte geografico verdadero. Para obtener el valor real visitar la web
134 // (https://www.ncei.noaa.gov/products/world-magnetic-model)
135
136 #define MAG_DECLINATION_DEG    1.95f
137
138 // Offset entre el "forward" del IMU y el "forward" de la PCB
139 // Si la IMU esta rotada respecto a la placa, el heading
140 // tendra un offset constante respecto al norte real.
141 // Tambien se puede usar para corregir errores constantes
142 // que no provienen de la orientacion de la PCB
143
144 #define HEADING_BODY_OFFSET_DEG    -5.0f
145
146 // Calibracion actitud
147 #define PITCH_MOUNT_OFFSET_DEG     0.6f
148 #define ROLL_MOUNT_OFFSET_DEG      0.2f
149
150
151 // NRF24L01
152 #define NRF_CHANNEL                108
153 #define NRF_DATA_RATE              RF24_250KBPS
154 #define NRF_PA_LEVEL               RF24_PA_HIGH
155 #define NRF_RETRY_DELAY            5
156 #define NRF_RETRY_COUNT            15
157
158 #define NRF_ADDR_TX                 "SENSR"
159 #define NRF_ADDR_RX                 "CNTRL"
160
161 #define TELEMETRY_RATE_HZ           20
162 #define TELEMETRY_INTERVAL_MS      (1000 / TELEMETRY_RATE_HZ)
163

```


164 **#endif**

Código 1.2: Constantes de hardware y parámetros de ajuste

```
1  // SensorData.h - estructuras de datos compartidas entre los archivos.
2
3  #ifndef SENSORDATA_H
4  #define SENSORDATA_H
5
6  // IMU sin procesar (MPU-9250 + AK8963)
7  struct RawIMU {
8      float ax, ay, az;    // acelerometro (g)
9      float gx, gy, gz;    // giroscopio (deg/s), ya con bias restado
10     float mx, my, mz;    // magnetometro (uT), ya con ASA aplicado
11 };
12
13 // AHRS ya fusionado
14 struct FlightData {
15     float pitch;          // -90 a +90deg (nariz arriba positivo)
16     float roll;           // -180 a +180deg (ala derecha abajo positivo)
17     float heading;        // 0 a 360deg (norte magnetico = 0)
18     float q0, q1, q2, q3; // cuaternion
19     unsigned long lastUpdate;
20 };
21
22 // Barometro (BMP280)
23 struct BaroData {
24     float pressure_hPa;
25     float temperature_C;
26     float altitude_m;      // altitud barometrica MSL
27     float vspeed_mps;      // velocidad vertical filtrada con EMA
28     bool ok;
29 };
30
31 // GPS (NEO-6M)
32 struct GPSData {
33     double lat;
34     double lon;
35     float altitude_m;
36     float speed_kmh;
37     float course_deg;
38     uint8_t satellites;
```

```

39     bool    fix;                // posicion valida
40     bool    ok;                // UART recibiendo datos
41 };
42
43 // Radar (HLK-LD2410C)
44 struct RadarData {
45     bool     presence;
46     bool     movingDetected;
47     uint16_t movingDist_cm;
48     uint8_t  movingEnergy;
49     bool     stationaryDetected;
50     uint16_t stationaryDist_cm;
51     uint8_t  stationaryEnergy;
52     bool     ok;
53     bool     platformStationary;
54     uint16_t bestDist_cm;
55     uint8_t  maxEnergy;
56     uint8_t  proximityZone;
57     bool     movingTargetAlert;
58 };
59
60 // Time-of-Flight (VL53L1X / TOF400C)
61 struct ToFData {
62     uint16_t distance_mm;
63     bool     valid;
64     bool     ok;
65 };
66
67 // Paquete de telemetria ARINC 429 sobre NRF24L01
68 //
69 // El paquete ocupa 32 bytes (8 palabras x 32 bits = payload maximo del
70 // NRF24L01) y se envia a 20 Hz.
71 struct ARINC429Packet {
72     uint32_t words[8];
73 };
74
75 // Slot vacio: una palabra con label 0 y SSM = NO_DATA, asi el receptor
76 // la ignora limpiamente. Lo usa el planificador cuando no hay nada que
77 // mandar en un slot
78 #define A429_EMPTY_WORD          0x20000000UL // SSM=01 (no data), todo cero
79
80 // Layout del campo discreto de la palabra de status (19 bits utiles)
81 #define A429_STATUS_SATS_MASK    0x0000F
82 #define A429_STATUS_FIX          0x00010

```

```

83 #define A429_STATUS_IMU_OK          0x00020
84 #define A429_STATUS_BARO_OK         0x00040
85 #define A429_STATUS_RADAR_OK        0x00080
86 #define A429_STATUS_TOF_OK          0x00100
87 #define A429_STATUS_RADAR_ZONE_SH   9
88 #define A429_STATUS_RADAR_ZONE_MASK 0x00600
89 #define A429_STATUS_RADAR_ALERT      0x00800
90 #define A429_STATUS_PLAT_MOVING      0x01000
91 #define A429_STATUS_SEQ_SH          13
92 #define A429_STATUS_SEQ_MASK        0x7E000
93
94 // Codificacion de la posicion GNSS en ARINC 429 BNR
95 // Valor de referencia para mejorar la medida
96 #define GPS_LAT_REF_DEG              41.0           // referencia de latitud
97 #define GPS_LON_REF_DEG              1.0           // referencia de longitud
98 #define A429_LSB_LATLON_DEG         (1.0f / 262144.0f) // ~3.81e-6 deg/LSB (~0.42
99                                     m)
100 #endif

```

Código 1.3: Estructuras de datos compartidas

```

1 // Calibration.ino - calibracion del bias del giroscopio al arranque.
2
3 // Bias del giroscopio
4 float gyroBiasX = 0, gyroBiasY = 0, gyroBiasZ = 0;
5
6 void calibrateGyro() {
7     Serial.println("CAL: Calibrando gyro NO MOVER EL SENSOR...");
8     neopixelWrite(PIN_LED_RGB, 30, 15, 0); // ambar mientras se calibra
9
10    float sumX = 0, sumY = 0, sumZ = 0;
11    float sumSqX = 0, sumSqY = 0, sumSqZ = 0;
12
13    for (int i = 0; i < CALIBRATION_SAMPLES; i++) {
14        float gx = readWord(MPU9250_ADDR, 0x43) / GYRO_SCALE;
15        float gy = readWord(MPU9250_ADDR, 0x45) / GYRO_SCALE;
16        float gz = readWord(MPU9250_ADDR, 0x47) / GYRO_SCALE;
17
18        sumX += gx; sumY += gy; sumZ += gz;
19        sumSqX += gx * gx;
20        sumSqY += gy * gy;

```

```
21     sumSqZ += gz * gz;
22
23     delay(10);
24 }
25
26 float meanX = sumX / CALIBRATION_SAMPLES;
27 float meanY = sumY / CALIBRATION_SAMPLES;
28 float meanZ = sumZ / CALIBRATION_SAMPLES;
29
30 // Varianza: Var = E[X^2] - E[X]^2
31 float varX = (sumSqX / CALIBRATION_SAMPLES) - (meanX * meanX);
32 float varY = (sumSqY / CALIBRATION_SAMPLES) - (meanY * meanY);
33 float varZ = (sumSqZ / CALIBRATION_SAMPLES) - (meanZ * meanZ);
34 float maxVariance = max(varX, max(varY, varZ));
35
36 Serial.print("CAL: Varianza X="); Serial.print(varX, 4);
37 Serial.print("  Y="); Serial.print(varY, 4);
38 Serial.print("  Z="); Serial.print(varZ, 4);
39 Serial.print("  (umbral="); Serial.print(CALIBRATION_VARIANCE_THRESHOLD, 1);
40 Serial.println(")");
41
42 if (maxVariance < CALIBRATION_VARIANCE_THRESHOLD) {
43     // Sensor en reposo durante toda la captura: la media es un bias fiable
44     gyroBiasX = meanX;
45     gyroBiasY = meanY;
46     gyroBiasZ = meanZ;
47
48     Serial.print("CAL: [LIVE] Bias X="); Serial.print(gyroBiasX, 4);
49     Serial.print("  Y="); Serial.print(gyroBiasY, 4);
50     Serial.print("  Z="); Serial.println(gyroBiasZ, 4);
51 } else {
52     // Movimiento detectado durante la calibracion. Como fallback se usan
53     // valores tipicos obtenidos previamente en prueba controlada, y el
54     // filtro mahony convergera al valor real mas rapido.
55     gyroBiasX = -0.2359;
56     gyroBiasY = 0.3017;
57     gyroBiasZ = -0.5030;
58     Serial.println("CAL: AVISO: movimiento detectado, usando bias por defecto")
59     ;
60 }
61
62 Serial.println("CAL: Calibracion del gyro completada.");
```

```
62 }
```

Código 1.4: Calibración del bias del giroscopio

```
1  // IMU.ino - lectura cruda del MPU-9250 (accel + gyro) y del AK8963 (mag).
2  // La fusion a Euler/quaternion se hace en SensorFusion.ino.
3
4
5  // Ajuste de sensibilidad de fabrica (ASA) del magnetometro
6  float magCalX = 1.0f, magCalY = 1.0f, magCalZ = 1.0f;
7
8  // Helpers I2C
9  void writeRegister(byte addr, byte reg, byte val) {
10     Wire.beginTransmission(addr);
11     Wire.write(reg);
12     Wire.write(val);
13     Wire.endTransmission();
14 }
15
16 int16_t readWord(byte addr, byte reg) {
17     Wire.beginTransmission(addr);
18     Wire.write(reg);
19     Wire.endTransmission(false);
20     Wire.requestFrom(addr, (byte)2);
21     uint8_t msb = Wire.read();
22     uint8_t lsb = Wire.read();
23     return (msb << 8) | lsb;
24 }
25
26 // Inicializacion
27 bool initIMU() {
28     // Sale del modo sleep
29     writeRegister(MPU9250_ADDR, 0x6B, 0x00);
30     delay(100);
31
32     // Comprueba WHO_AM_I - debe ser 0x71
33     Wire.beginTransmission(MPU9250_ADDR);
34     Wire.write(0x75);
35     Wire.endTransmission(false);
36     Wire.requestFrom(MPU9250_ADDR, (byte)1);
37     byte whoAmI = Wire.read();
38 }
```

```
39  if (whoAmI != 0x71) {
40      Serial.print("IMU ERROR: WHO_AM_I = 0x");
41      Serial.println(whoAmI, HEX);
42      return false;
43  }
44
45  writeRegister(MPU9250_ADDR, 0x1C, 0x08); // Accel +/-4g
46  writeRegister(MPU9250_ADDR, 0x1B, 0x08); // Gyro +/-500deg/s
47  writeRegister(MPU9250_ADDR, 0x1A, 0x03); // DLPF ~42 Hz
48  writeRegister(MPU9250_ADDR, 0x37, 0x02); // bypass I2C para poder hablar con
    el AK8963
49  delay(100);
50
51  // Comprueba la ID del magnetometro (WIA = 0x48)
52  Wire.beginTransmission(AK8963_ADDR);
53  Wire.write(0x00);
54  Wire.endTransmission(false);
55  Wire.requestFrom(AK8963_ADDR, (byte)1);
56  byte magId = Wire.read();
57
58  if (magId != 0x48) {
59      Serial.print("MAG ERROR: WIA = 0x");
60      Serial.println(magId, HEX);
61      return false;
62  }
63
64  // Lee el ASA desde la Fuse ROM del AK8963 para corregir la sensibilidad
65  // de cada eje. Formula del datasheet MS1356-E-02 sec.8.3.11.
66  writeRegister(AK8963_ADDR, 0x0A, 0x00); // power down
67  delay(10);
68  writeRegister(AK8963_ADDR, 0x0A, 0x0F); // modo Fuse ROM
69  delay(10);
70
71  Wire.beginTransmission(AK8963_ADDR);
72  Wire.write(0x10); // registro ASAX
73  Wire.endTransmission(false);
74  Wire.requestFrom(AK8963_ADDR, (byte)3);
75
76  uint8_t asaX = Wire.read();
77  uint8_t asaY = Wire.read();
78  uint8_t asaZ = Wire.read();
79
80  magCalX = ((float)(asaX - 128) / 256.0f) + 1.0f;
81  magCalY = ((float)(asaY - 128) / 256.0f) + 1.0f;
```

```
82  magCalZ = ((float)(asaZ - 128) / 256.0f) + 1.0f;
83
84  Serial.print("MAG ASA: X="); Serial.print(asaX);
85  Serial.print("  Y="); Serial.print(asaY);
86  Serial.print("  Z="); Serial.println(asaZ);
87  Serial.print("MAG Cal: X="); Serial.print(magCalX, 4);
88  Serial.print("  Y="); Serial.print(magCalY, 4);
89  Serial.print("  Z="); Serial.println(magCalZ, 4);
90
91  writeRegister(AK8963_ADDR, 0x0A, 0x00);  // power down otra vez para
      reconfigurar
92  delay(10);
93  writeRegister(AK8963_ADDR, 0x0A, 0x16);  // 16-bit, continuo a 100 Hz
94  delay(100);
95
96  Serial.println("IMU: MPU-9250 + AK8963 OK");
97
98  return true;
99 }
100
101 // Lectura de los 9 ejes
102 void readIMU(RawIMU &imu) {
103   // Burst de 14 bytes desde 0x3B: accel (6) + temp (2) + gyro (6)
104   Wire.beginTransaction(MPU9250_ADDR);
105   Wire.write(0x3B);
106   Wire.endTransmission(false);
107   Wire.requestFrom(MPU9250_ADDR, (byte)14);
108
109   // MPU-9250 es big-endian (MSB primero, LSB despues)
110   uint8_t msb, lsb;
111   msb = Wire.read(); lsb = Wire.read();
112   int16_t axRaw = (msb << 8) | lsb;
113   msb = Wire.read(); lsb = Wire.read();
114   int16_t ayRaw = (msb << 8) | lsb;
115   msb = Wire.read(); lsb = Wire.read();
116   int16_t azRaw = (msb << 8) | lsb;
117
118   // La temperatura queda entre accel y gyro. No se usa pero hay que
119   // leerla igual porque va en el mismo paquete
120   msb = Wire.read(); lsb = Wire.read();
121   int16_t tempRaw = (msb << 8) | lsb;
122   (void)tempRaw;
123
124   msb = Wire.read(); lsb = Wire.read();
```

```

125  int16_t gxRaw = (msb << 8) | lsb;
126  msb = Wire.read(); lsb = Wire.read();
127  int16_t gyRaw = (msb << 8) | lsb;
128  msb = Wire.read(); lsb = Wire.read();
129  int16_t gzRaw = (msb << 8) | lsb;
130
131  imu.ax = axRaw / ACCEL_SCALE;
132  imu.ay = ayRaw / ACCEL_SCALE;
133  imu.az = azRaw / ACCEL_SCALE;
134
135  imu.gx = (gxRaw / GYRO_SCALE) - gyroBiasX;
136  imu.gy = (gyRaw / GYRO_SCALE) - gyroBiasY;
137  imu.gz = (gzRaw / GYRO_SCALE) - gyroBiasZ;
138
139  // AK8963: 7 bytes desde 0x03 (6 de datos + ST2). El ST2 hay que leerlo
140  // siempre para que el mag prepare la siguiente muestra
141  Wire.beginTransaction(AK8963_ADDR);
142  Wire.write(0x03);
143  Wire.endTransmission(false);
144  Wire.requestFrom(AK8963_ADDR, (byte)7);
145
146  // El AK8963 es little-endian (LSB primero, MSB despues)
147  uint8_t l, h;
148  l = Wire.read(); h = Wire.read();
149  int16_t mxRaw = l | (h << 8);
150  l = Wire.read(); h = Wire.read();
151  int16_t myRaw = l | (h << 8);
152  l = Wire.read(); h = Wire.read();
153  int16_t mzRaw = l | (h << 8);
154  uint8_t st2 = Wire.read(); // ST2, obligatorio leerlo
155  (void)st2;
156
157  // Aplicacion correcciones
158  float mxUT = mxRaw * MAG_SCALE * magCalX;
159  float myUT = myRaw * MAG_SCALE * magCalY;
160  float mzUT = mzRaw * MAG_SCALE * magCalZ;
161
162  imu.mx = (mxUT - MAG_OFFSET_X) * MAG_SCALE_X;
163  imu.my = (myUT - MAG_OFFSET_Y) * MAG_SCALE_Y;
164  imu.mz = (mzUT - MAG_OFFSET_Z) * MAG_SCALE_Z;
165  }

```

Código 1.5: Driver del MPU-9250 y AK8963


```
1  // Barometer.ino - driver del BMP280 (presion + temperatura + altitud MSL)
2
3  #include <Adafruit_BMP280.h>
4
5  Adafruit_BMP280 bmp;
6
7  // Estado para calcular la velocidad vertical por diferencias finitas
8  static float prevAltitude = 0.0f;
9  static unsigned long prevBaroTime = 0;
10
11 bool initBarometer() {
12     if (!bmp.begin(BMP280_ADDR)) {
13         Serial.println("BARO ERROR: BMP280 no encontrado en 0x76");
14         Serial.println("  Revisar: VCC=3.3V, SDA=GPI08, SCL=GPI09, SDO=GND");
15         return false;
16     }
17
18     // Configuracion para 10 Hz de lectura (BARO_INTERVAL_MS)
19     // Parametros segun BMP280 datasheet Bosch BST-BMP280-DS001-19 sec.3.5.
20     bmp.setSampling(
21         Adafruit_BMP280::MODE_NORMAL,
22         Adafruit_BMP280::SAMPLING_X2,
23         Adafruit_BMP280::SAMPLING_X16,
24         Adafruit_BMP280::FILTER_X16,
25         Adafruit_BMP280::STANDBY_MS_63
26     );
27
28     // Base para el calculo de velocidad vertical
29     prevAltitude = bmp.readAltitude(SEA_LEVEL_HPA);
30     prevBaroTime = millis();
31
32     Serial.println("BARO: BMP280 OK");
33     Serial.print("  Altitud inicial: ");
34     Serial.print(prevAltitude, 1);
35     Serial.println(" m MSL");
36
37     return true;
38 }
39
40 void readBarometer(BaroData &baro) {
41     baro.pressure_hPa = bmp.readPressure() / 100.0f;  // Pa -> hPa
42     baro.temperature_C = bmp.readTemperature();
43     baro.altitude_m = bmp.readAltitude(SEA_LEVEL_HPA);
```

```

44
45 // Velocidad vertical por diferencia finita, suavizada con EMA.
46 // El IIR interno del BMP280 reduce el ruido de presion pero queda
47 // la discretizacion del LSB (~0.16 Pa). Sin el EMA el VSI se ve
48 // saltar en pasos discretos al derivar.
49 unsigned long now = millis();
50 float dt = (now - prevBaroTime) / 1000.0f;
51
52 if (dt > 0.05f && dt < 2.0f) { // protege contra div/0 y dt anomalos
53     float rawVS = (baro.altitude_m - prevAltitude) / dt;
54     baro.vspeed_mps = baro.vspeed_mps * (1.0f - VSPEED_ALPHA)
55         + rawVS * VSPEED_ALPHA;
56 }
57
58 prevAltitude = baro.altitude_m;
59 prevBaroTime = now;
60 }

```

Código 1.6: Driver del barómetro BMP280

```

1 // GPS.ino - driver del NEO-6M (UART1) con TinyGPSPlus
2
3 #include <TinyGPSPlus.h>
4
5 TinyGPSPlus gps;
6 HardwareSerial GPS_Serial(1); // UART1
7
8 void initGPS() {
9     GPS_Serial.begin(GPS_BAUD, SERIAL_8N1, GPS_RX_PIN, GPS_TX_PIN);
10
11     Serial.print("GPS: UART1 en RX=GPIO");
12     Serial.print(GPS_RX_PIN);
13     Serial.print(", TX=GPIO");
14     Serial.print(GPS_TX_PIN);
15     Serial.print(", Baud=");
16     Serial.println(GPS_BAUD);
17
18     // Chequeo rapido: esta llegando algo por el puerto?
19     unsigned long start = millis();
20     bool dataReceived = false;
21     while (millis() - start < 1500) {
22         if (GPS_Serial.available()) {

```

```
23     dataReceived = true;
24     break;
25 }
26 }
27
28 if (dataReceived) {
29     Serial.println("GPS: NMEA detectada -- OK");
30 } else {
31     Serial.println("GPS: AVISO -- aun no llega NMEA (suele necesitar cielo
32     abierto)");
33 }
34 }
35
36 // TinyGPSPlus necesita recibir los bytes en cuanto llegan. Esta funcion se
37 // llama cada iteracion del loop para que no se pierda nada en el buffer
38 // de la UART.
39 void feedGPS() {
40     while (GPS_Serial.available() > 0) {
41         gps.encode(GPS_Serial.read());
42     }
43 }
44
45 void readGPS(GPSData &data) {
46     feedGPS();
47
48     // "ok" = UART recibe datos (aunque no haya fix)
49     data.ok = (gps.charsProcessed() > 10);
50
51     // Posicion
52     if (gps.location.isValid()) {
53         data.lat = gps.location.lat();
54         data.lon = gps.location.lng();
55         data.fix = true;
56     } else {
57         data.fix = false;
58     }
59
60     // Solo se actualiza el resto de campos si hay dato valido, asi los
61 // ultimos valores conocidos se mantienen si hay algun problema
62     if (gps.altitude.isValid()) data.altitude_m = gps.altitude.meters();
63     if (gps.speed.isValid()) data.speed_kmh = gps.speed.kmph();
64     if (gps.course.isValid()) data.course_deg = gps.course.deg();
65     if (gps.satellites.isValid()) data.satellites = gps.satellites.value();
```

```
66 }
```

Código 1.7: Driver del GPS NEO-6M

```
1  // ToF.ino - driver del VL53L1X (TOF400C)
2
3  #include <VL53L1X.h>
4
5  VL53L1X tofSensor;
6
7  bool initToF() {
8      tofSensor.setTimeout(500);
9
10     if (!tofSensor.init()) {
11         Serial.println("TOF ERROR: VL53L1X no encontrado");
12         Serial.println("  Revisar: VCC=3.3V, SDA=GPI08, SCL=GPI09, XSHUT en alto");
13         return false;
14     }
15
16     tofSensor.setDistanceMode(VL53L1X::Long);           // hasta 4 m
17     tofSensor.setMeasurementTimingBudget(TOF_TIMING_BUDGET_US);
18     tofSensor.startContinuous(TOF_CONTINUOUS_MS);
19
20     Serial.println("TOF: VL53L1X OK");
21     return true;
22 }
23
24 // Lectura no bloqueante. Si no hay muestra nueva, no se toca data.
25 void readToF(ToFData &data) {
26     if (!data.ok) return;
27
28     if (!tofSensor.dataReady()) return;
29
30     uint16_t dist = tofSensor.read(false); // false = no bloquear
31
32     if (tofSensor.timeoutOccurred()) {
33         data.valid = false;
34         return;
35     }
36
37     data.distance_mm = dist;
38     data.valid = (dist >= TOF_DEAD_ZONE_MM);
```

```
39 }
```

Código 1.8: Driver del VL53L1X (Time-of-Flight)

```
1  // Radar.ino - driver del HLK-LD2410C (radar 24 GHz FMCW) en UART2.
2
3  #include <MyLD2410.h>
4
5  HardwareSerial RadarSerial(2);  // UART2
6  MyLD2410 radar(RadarSerial);
7
8  // Suavizado EMA de la distancia
9  static float smoothedDist_cm = 0.0f;
10 static bool hasSmoothedDist = false;
11
12 bool initRadar() {
13     RadarSerial.begin(RADAR_BAUD, SERIAL_8N1, RADAR_RX_PIN, RADAR_TX_PIN);
14
15     Serial.print("RADAR: UART2 en RX=GPIO");
16     Serial.print(RADAR_RX_PIN);
17     Serial.print(", TX=GPIO");
18     Serial.print(RADAR_TX_PIN);
19     Serial.print(", Baud=");
20     Serial.println(RADAR_BAUD);
21
22     if (!radar.begin()) {
23         Serial.println("RADAR: AVISO -- handshake LD2410C fallo");
24         Serial.println(" Se reintentara en check() desde el loop principal");
25         return false;
26     }
27
28     Serial.println("RADAR: LD2410C OK");
29
30     // Firmware
31     if (radar.requestFirmware()) {
32         Serial.print(" Firmware: ");
33         Serial.println(radar.getFirmware());
34     }
35
36     // Configuracion
37     // Las setters requieren el radar en modo configuracion. enhancedMode()
38     // y resetParameters() lo gestionan ellos solos, pero
```

```
39 // setGateParameters() necesita configMode(true) explicito.
40 if (!radar.configMode(true)) {
41     Serial.println(" AVISO: no se pudo entrar en modo config");
42 }
43
44 // Resolucion del gate
45 // Modo fino (20 cm/gate, 8 gates, max 1.8 m). La libreria usa
46 // true = 0.20 m/gate, false = 0.75 m/gate (segun datasheet
47 // LD2410C v1.05 sec.2.2). Para esta aplicacion el rango util
48 // es el aterrizaje, donde 1.8 m es suficiente y la resolucion
49 // fina es prioritaria sobre el alcance.
50 if (radar.requestResolution()) {
51     if (radar.getResolution() != RADAR_GATE_WIDTH_CM) {
52         Serial.println(" Resolucion: cambiando a 20 cm/gate");
53         radar.setResolution(true); // true = 20 cm, false = 75 cm
54         radar.requestReboot();
55         delay(1500); // espera al reboot del radar
56         radar.begin();
57         radar.configMode(true);
58     } else {
59         Serial.print(" Resolucion: ");
60         Serial.print(radar.getResolution());
61         Serial.println(" cm/gate (max 1.8 m)");
62     }
63 }
64
65 // Thresholds por gate ajustadas a downward-facing.
66 // Gates cercanos: thresholds ALTAS para no saturar con clutter del
67 // chasis y vibracion. Gates lejanos: thresholds BAJAS para detectar
68 // el suelo sin perderlo a mas distancia.
69 radar.setGateParameters(0, RADAR_SENS_GATE_0, RADAR_SENS_GATE_0);
70 radar.setGateParameters(1, RADAR_SENS_GATE_1, RADAR_SENS_GATE_1);
71 radar.setGateParameters(2, RADAR_SENS_GATE_2, RADAR_SENS_GATE_2);
72 radar.setGateParameters(3, RADAR_SENS_GATE_3, RADAR_SENS_GATE_3);
73 for (uint8_t g = 4; g <= 8; g++) {
74     radar.setGateParameters(g, RADAR_SENS_GATE_FAR, RADAR_SENS_GATE_FAR);
75 }
76 Serial.println(" Thresholds por gate: configuradas");
77
78 radar.configMode(false);
79
80 // Modo enhanced
81 if (radar.enhancedMode(true)) {
82     Serial.println(" Modo enhanced: ON (per-gate signals disponibles)");
```

```
83     } else {
84         Serial.println("  Modo enhanced: OFF (solo basic mode)");
85     }
86
87     return true;
88 }
89
90 // Procesa los bytes que hayan llegado por UART. Llamar cada iteracion
91 // del loop para no perder paquetes
92 void feedRadar() {
93     radar.check();
94 }
95
96 // Lee datos y calcula las senales derivadas
97 // platformSpeed_kmh: velocidad GPS actual, necesaria para decidir si
98 // se puede confiar en la clasificacion movil/estacionario.
99 void readRadar(RadarData &data, float platformSpeed_kmh) {
100     feedRadar();
101
102     // ok = el radar envia frames
103     data.ok = (radar.getFrameCount() > 0);
104     if (!data.ok) return;
105
106     // Datos del algoritmo interno del radar
107     data.presence = (radar.getStatus() != 0);
108
109     if (radar.movingTargetDetected()) {
110         data.movingDetected = true;
111         data.movingDist_cm = (uint16_t)radar.movingTargetDistance();
112         data.movingEnergy = radar.movingTargetSignal();
113     } else {
114         data.movingDetected = false;
115         data.movingDist_cm = 0;
116         data.movingEnergy = 0;
117     }
118
119     if (radar.stationaryTargetDetected()) {
120         data.stationaryDetected = true;
121         data.stationaryDist_cm = (uint16_t)radar.stationaryTargetDistance();
122         data.stationaryEnergy = radar.stationaryTargetSignal();
123     } else {
124         data.stationaryDetected = false;
125         data.stationaryDist_cm = 0;
126         data.stationaryEnergy = 0;
```

```

127     }
128
129     // La clasificacion Doppler solo es fiable con la plataforma parada
130     data.platformStationary = (platformSpeed_kmh < RADAR_HOVER_SPEED_KMH);
131
132     // Mejor estimacion de distancia
133     // Tomamos el target con mayor signal (el reflector dominante).
134     // Cuando el drone esta parado, lo normal es que el suelo sea
135     // stationary; cuando se mueve, el suelo aparece como moving por el
136     // Doppler shift. Asi que mirar a ambos cubre los dos casos.
137     uint16_t dist_cm = 0;
138     uint8_t  energy  = 0;
139     bool     valid   = false;
140
141     if (data.stationaryDetected && data.movingDetected) {
142         // Los dos a la vez: gana el de mas energia
143         if (data.stationaryEnergy >= data.movingEnergy) {
144             dist_cm = data.stationaryDist_cm;
145             energy  = data.stationaryEnergy;
146         } else {
147             dist_cm = data.movingDist_cm;
148             energy  = data.movingEnergy;
149         }
150     } else if (data.stationaryDetected) {
151         dist_cm = data.stationaryDist_cm;
152         energy  = data.stationaryEnergy;
153     } else if (data.movingDetected) {
154         dist_cm = data.movingDist_cm;
155         energy  = data.movingEnergy;
156     }
157
158     // Filtro adicional sobre la energia: el algoritmo del chip ya rechaza
159     // mucho ruido, pero a veces declara un target valido con signal bajo
160     // (en torno a 20-30) que es solo eco residual. Con RADAR_NOISE_FLOOR
161     // descartamos esos casos para que la salida sea estable.
162     valid = (dist_cm > 0) && (energy >= RADAR_NOISE_FLOOR);
163
164     if (valid) {
165         // Suavizado EMA. Reduce el flicker frame-a-frame que aparece
166         // cuando el chip alterna entre dos gates adyacentes
167         if (!hasSmoothedDist) {
168             smoothedDist_cm = (float)dist_cm;
169             hasSmoothedDist = true;
170         } else {

```



```

171     smoothedDist_cm = smoothedDist_cm * (1.0f - RADAR_DIST_ALPHA)
172         + (float)dist_cm * RADAR_DIST_ALPHA;
173 }
174 data.bestDist_cm = (uint16_t)(smoothedDist_cm + 0.5f);
175 data.maxEnergy    = energy;
176 } else {
177     // Sin target valido: se resetea el filtro para que el siguiente
178     // eco arranque limpio
179     data.bestDist_cm = 0;
180     data.maxEnergy    = 0;
181     hasSmoothedDist   = false;
182 }
183
184 // Zona de proximidad en base a la distancia con histeresis implicita
185 // por el suavizado EMA
186 data.proximityZone = RADAR_ZONE_NONE;
187 if (data.bestDist_cm > 0 && data.maxEnergy >= RADAR_ALERT_ENERGY_MIN) {
188     if (data.bestDist_cm <= 60)         data.proximityZone = RADAR_ZONE_DANGER;
189     else if (data.bestDist_cm <= 120)   data.proximityZone = RADAR_ZONE_CAUTION;
190     else                               data.proximityZone = RADAR_ZONE_CLEAR;
191 }
192
193 // Target movil en zona de aterrizaje. Solo se evalua con la
194 // plataforma parada; en movimiento la clasificacion Doppler del
195 // chip ve el suelo como "movil" y dispararia falsos positivos.
196 data.movingTargetAlert = false;
197 if (data.platformStationary && data.movingDetected) {
198     if (data.movingDist_cm <= 120 && data.movingEnergy >=
199         RADAR_ALERT_ENERGY_MIN) {
200         data.movingTargetAlert = true;
201     }
202 }

```

Código 1.9: Driver del radar HLK-LD2410C

```

1 // SensorFusion.ino - Filtro MARG de Mahony (9 ejes).
2
3
4 // Estado del filtro
5 static float q0 = 1.0f, q1 = 0.0f, q2 = 0.0f, q3 = 0.0f;           //
    cuaternion

```

```

6 static float integralFBx = 0.0f, integralFBy = 0.0f, integralFBz = 0.0f; //
   acumulador Ki
7
8 void initFusion(FlightData &data) {
9     q0 = 1.0f; q1 = 0.0f; q2 = 0.0f; q3 = 0.0f;
10    integralFBx = 0.0f; integralFBy = 0.0f; integralFBz = 0.0f;
11
12    data.pitch = 0; data.roll = 0; data.heading = 0;
13    data.q0 = q0; data.q1 = q1; data.q2 = q2; data.q3 = q3;
14    data.lastUpdate = micros();
15
16    Serial.println("FUSION: filtro Mahony MARG inicializado");
17    Serial.print("FUSION: Kp="); Serial.print(MAHONY_KP, 2);
18    Serial.print(" Ki="); Serial.println(MAHONY_KI, 2);
19    Serial.print("FUSION: declinacion="); Serial.print(MAG_DECLINATION_DEG, 2);
20    Serial.print(" body offset="); Serial.println(HEADING_BODY_OFFSET_DEG, 2);
21 }
22
23 static float invSqrt(float x) {
24     return 1.0f / sqrtf(x);
25 }
26
27 void updateFusion(RawIMU &imu, FlightData &data) {
28     unsigned long now = micros();
29     float dt = (now - data.lastUpdate) / 1000000.0f;
30     data.lastUpdate = now;
31     // Proteccion: dt negativo (overflow) o enorme (sleep) -> se salta un ciclo
32     if (dt <= 0.0f || dt > 0.5f) return;
33
34     // Gyro a rad/s
35     float gx = imu.gx * 0.0174533f;
36     float gy = imu.gy * 0.0174533f;
37     float gz = imu.gz * 0.0174533f;
38
39     // Remap del magnetometro al frame del MPU (accel/gyro).
40     // El AK8963 tiene los ejes orientados diferente respecto al accel/gyro
41     // del MPU (los dos chips estan rotados internamente). Remap segun
42     // PS-MPU-9250A-01 sec.9.1, validado por la libreria de referencia
43     // de Kris Winer.
44     float mx = imu.my;
45     float my = imu.mx;
46     float mz = -imu.mz;
47
48     // Normaliza el accel

```

```

49 float ax = imu.ax, ay = imu.ay, az = imu.az;
50 float recipNorm = invSqrt(ax * ax + ay * ay + az * az);
51 bool useAccel = (recipNorm < 10.0f); // si la norma es ~0 lo ignoramos
52 if (useAccel) {
53     ax *= recipNorm;
54     ay *= recipNorm;
55     az *= recipNorm;
56 }
57
58 // Normaliza el mag. Si la norma no es logica (desconectado o saturado),
59 // se ignora al mag en este ciclo y el filtro utiliza solo gyro+accel.
60 bool useMag = true;
61 recipNorm = invSqrt(mx * mx + my * my + mz * mz);
62 if (recipNorm > 100.0f || recipNorm < 0.001f) {
63     useMag = false;
64 } else {
65     mx *= recipNorm;
66     my *= recipNorm;
67     mz *= recipNorm;
68 }
69
70 // Nucleo de Mahony
71
72 // Productos precalculados del cuaternion
73 float q0q0 = q0 * q0, q0q1 = q0 * q1, q0q2 = q0 * q2, q0q3 = q0 * q3;
74 float q1q1 = q1 * q1, q1q2 = q1 * q2, q1q3 = q1 * q3;
75 float q2q2 = q2 * q2, q2q3 = q2 * q3;
76 float q3q3 = q3 * q3;
77
78 float ex = 0.0f, ey = 0.0f, ez = 0.0f; // error acumulado
79
80 // Correccion con el magnetometro
81 if (useMag) {
82     // Rota el mag medido al frame Earth
83     float hx = 2.0f * (mx * (0.5f - q2q2 - q3q3) + my * (q1q2 - q0q3) + mz * (
84         q1q3 + q0q2));
85     float hy = 2.0f * (mx * (q1q2 + q0q3) + my * (0.5f - q1q1 - q3q3) + mz * (
86         q2q3 - q0q1));
87     float hz = 2.0f * (mx * (q1q3 - q0q2) + my * (q2q3 + q0q1) + mz * (0.5f -
88         q1q1 - q2q2));
89
90     // Referencia [bx, 0, bz]: se elimina la componente Y para que el
91     // heading no influya en el error, solo la inclinacion
92     float bx = sqrtf(hx * hx + hy * hy);

```

```

90     float bz = hz;
91
92     // Mag esperado en body frame: w = q* (x) [0,bx,0,bz] (x) q
93     float wx = 2.0f * (bx * (0.5f - q2q2 - q3q3) + bz * (q1q3 - q0q2));
94     float wy = 2.0f * (bx * (q1q2 - q0q3) + bz * (q0q1 + q2q3));
95     float wz = 2.0f * (bx * (q0q2 + q1q3) + bz * (0.5f - q1q1 - q2q2));
96
97     // Error = producto vectorial del mag medido vs esperado
98     ex += (my * wz - mz * wy);
99     ey += (mz * wx - mx * wz);
100    ez += (mx * wy - my * wx);
101 }
102
103 // Correccion con el accel
104 if (useAccel) {
105     // Direccion de la gravedad esperada en body frame (segun el cuaternion)
106     float vx = 2.0f * (q1q3 - q0q2);
107     float vy = 2.0f * (q0q1 + q2q3);
108     float vz = q0q0 - q1q1 - q2q2 + q3q3;
109
110     // Error = producto vectorial gravedad medida vs esperada
111     ex += (ay * vz - az * vy);
112     ey += (az * vx - ax * vz);
113     ez += (ax * vy - ay * vx);
114 }
115
116 // Controlador PI
117 // Ki integra el error y estima online el bias residual del gyro
118 if (MAHONY_KI > 0.0f) {
119     integralFBx += MAHONY_KI * ex * dt;
120     integralFBy += MAHONY_KI * ey * dt;
121     integralFBz += MAHONY_KI * ez * dt;
122     gx += integralFBx;
123     gy += integralFBy;
124     gz += integralFBz;
125 }
126
127 // Termino proporcional
128 gx += MAHONY_KP * ex;
129 gy += MAHONY_KP * ey;
130 gz += MAHONY_KP * ez;
131
132 // Integracion del cuaternion
133 // Se guarda el estado completo antes de integrar para no mezclar

```

```

134 // componentes ya actualizados con los que aun no lo estan.
135 float qa = q0, qb = q1, qc = q2, qd = q3;
136
137 q0 += (-qb * gx - qc * gy - qd * gz) * (0.5f * dt);
138 q1 += ( qa * gx + qc * gz - qd * gy) * (0.5f * dt);
139 q2 += ( qa * gy - qb * gz + qd * gx) * (0.5f * dt);
140 q3 += ( qa * gz + qb * gy - qc * gx) * (0.5f * dt);
141
142 // Renormalizacion, evita que el cuaternion se aleje del manifold unitario
143 recipNorm = invSqrt(q0 * q0 + q1 * q1 + q2 * q2 + q3 * q3);
144 q0 *= recipNorm; q1 *= recipNorm; q2 *= recipNorm; q3 *= recipNorm;
145
146 // Cuaternion -> angulos de Euler (convencion ZYX, frame MPU nativo)
147 float sinPitch = 2.0f * (q0 * q2 - q3 * q1);
148 sinPitch = constrain(sinPitch, -1.0f, 1.0f); // evita NaN si hay ruido
        numerico
149
150 // Conversion a frame aeronautico NED (X forward, Y right, Z down).
151 // - Pitch (sobre Y): se niega
152 // - Roll  (sobre X): no cambia
153 // - Yaw   (sobre Z): se niega
154 data.pitch = -(asin(sinPitch) * 57.2958f) - PITCH_MOUNT_OFFSET_DEG;
155 data.roll  = atan2f(2.0f * (q0 * q1 + q2 * q3),
156                  1.0f - 2.0f * (q1 * q1 + q2 * q2)) * 57.2958f -
        ROLL_MOUNT_OFFSET_DEG;
157
158 float yaw = -(atan2f(2.0f * (q0 * q3 + q1 * q2),
159                  1.0f - 2.0f * (q2 * q2 + q3 * q3)) * 57.2958f);
160
161 // Yaw [-180, +180] -> heading [0, 360], con dos correcciones:
162 // 1. MAG_DECLINATION_DEG
163 // 2. HEADING_BODY_OFFSET_DEG
164 yaw += MAG_DECLINATION_DEG;
165 yaw += HEADING_BODY_OFFSET_DEG;
166 if (yaw < 0.0f) yaw += 360.0f;
167 if (yaw >= 360.0f) yaw -= 360.0f;
168 data.heading = yaw;
169
170 data.q0 = q0; data.q1 = q1; data.q2 = q2; data.q3 = q3;
171 }
172
173
174 // Inicializa el cuaternion a partir de una sola muestra de accel + mag
175 // Sin este valor el filtro Mahony arranca con

```

```

176 // cuaternion identidad y necesita varios segundos para converger
177 void seedQuaternionFromIMU(RawIMU &imu) {
178     // Pitch/roll desde el accel asumiendo gravedad pura
179     float ax = imu.ax, ay = imu.ay, az = imu.az;
180     float roll = atan2f(ay, az);
181     float pitch = atan2f(-ax, sqrtf(ay*ay + az*az));
182
183     // Mismo remap del magnetometro que en updateFusion()
184     float mx = imu.my;
185     float my = imu.mx;
186     float mz = -imu.mz;
187
188     // Heading compensado por inclinacion: proyecta el vector magnetico
189     // al plano horizontal antes de extraer el angulo
190     float cr = cosf(roll), sr = sinf(roll);
191     float cp = cosf(pitch), sp = sinf(pitch);
192     float mxh = mx * cp + my * sr * sp + mz * cr * sp;
193     float myh = my * cr - mz * sr;
194     float yaw = atan2f(-myh, mxh); // signo segun la convencion NED del filtro
195
196     // Conversion Euler ZYX -> cuaternion
197     float cy = cosf(yaw * 0.5f), sy = sinf(yaw * 0.5f);
198     float cp2 = cosf(pitch * 0.5f), sp2 = sinf(pitch * 0.5f);
199     float cr2 = cosf(roll * 0.5f), sr2 = sinf(roll * 0.5f);
200
201     q0 = cr2*cp2*cy + sr2*sp2*sy;
202     q1 = sr2*cp2*cy - cr2*sp2*sy;
203     q2 = cr2*sp2*cy + sr2*cp2*sy;
204     q3 = cr2*cp2*sy - sr2*sp2*cy;
205 }

```

Código 1.10: Filtro Mahony MARG de 9 ejes

```

1 // Telemetry.ino - transmisor NRF24L01+PA+LNA con payload ARINC 429 y
2 // planificador de tasas por etiqueta.
3
4 #include <SPI.h>
5 #include <RF24.h>
6 #include <ARINC429.h>
7
8 RF24 radio(NRF_CE, NRF_CSN);
9 const byte addrTX[6] = NRF_ADDR_TX;

```

```

10
11 uint32_t txCount = 0;
12 uint32_t txFailCount = 0;
13 static uint8_t txSeqNumber = 0;
14
15 // Slots del planificador. Los slots 0-2 (pitch, roll, heading) son
16 // fijos y van en cada paquete (20 Hz). Los slots 3-7 multiplexan los
17 // parametros listados aqui segun su periodo nominal
18 enum SchedSlot {
19     SCH_PRES_ALT = 0,
20     SCH_VSPEED,
21     SCH_RADAR_DIST,
22     SCH_RADIO_ALT,
23     SCH_GPS_ALT,
24     SCH_LATITUDE,
25     SCH_LONGITUDE,
26     SCH_STATUS,
27     SCH_COUNT
28 };
29
30 // Periodo nominal en ms entre dos transmisiones del mismo parametro
31 static const uint16_t schedPeriodMs[SCH_COUNT] = {
32     100,    // PRES_ALT    (10 Hz)
33     100,    // VSPEED      (10 Hz)
34     100,    // RADAR        (10 Hz)
35     100,    // RADIO_ALT    (10 Hz)
36     200,    // GPS_ALT      (5 Hz)
37     200,    // LATITUDE     (5 Hz)
38     200,    // LONGITUDE    (5 Hz)
39     200,    // STATUS       (5 Hz)
40 };
41
42 static unsigned long schedLastSent[SCH_COUNT] = {0};
43
44 bool initTelemetry() {
45     SPI.begin(NRF_SCK, NRF_MISO, NRF_MOSI, -1);
46     if (!radio.begin(&SPI)) {
47         Serial.println("RADIO ERROR: NRF24L01 no detectado");
48         return false;
49     }
50
51     radio.setPALevel(NRF_PA_LEVEL);
52     radio.setDataRate(NRF_DATA_RATE);
53     radio.setChannel(NRF_CHANNEL);

```

```

54  radio.setRetries(NRF_RETRY_DELAY, NRF_RETRY_COUNT);
55  radio.setPayloadSize(sizeof(ARINC429Packet));
56
57  radio.openWritingPipe(addrTX);
58  radio.stopListening();
59
60  Serial.println("RADIO: NRF24L01+PA+LNA OK (payload ARINC 429)");
61  Serial.print("  Canal: ");   Serial.print(radio.getChannel());
62  Serial.print("  Payload: "); Serial.print(sizeof(ARINC429Packet));
63  Serial.print("B (8 palabras x 32 bits)  Rate: ");
64  Serial.print(TELEMETRY_RATE_HZ);
65  Serial.println(" Hz");
66  Serial.println("  Planificador:");
67  Serial.println("    pitch/roll/hdg=20Hz  baro/radar/tof=10Hz");
68  Serial.println("    gps/status=5Hz");
69
70  return true;
71 }
72
73 // Construye el campo discreto del status word
74 static uint32_t buildStatusDiscrete(GPSData &gps, BaroData &baro, RadarData &
    radarData, ToFData &tofData, uint8_t seq) {
75  uint32_t d = 0;
76  uint8_t sats = (gps.satellites > 15) ? 15 : gps.satellites;
77  d |= (sats & A429_STATUS_SATS_MASK);
78  if (gps.fix)          d |= A429_STATUS_FIX;
79  if (baro.ok)          d |= A429_STATUS_BARO_OK;
80  if (radarData.ok)     d |= A429_STATUS_RADAR_OK;
81  if (tofData.ok)       d |= A429_STATUS_TOF_OK;
82
83  d |= ((uint32_t)(radarData.proximityZone & 0x3) << A429_STATUS_RADAR_ZONE_SH)
    ;
84  if (radarData.movingTargetAlert) d |= A429_STATUS_RADAR_ALERT;
85  if (!radarData.platformStationary) d |= A429_STATUS_PLAT_MOVING;
86
87  d |= ((uint32_t)(seq & 0x3F) << A429_STATUS_SEQ_SH);
88  return d;
89 }
90
91 // Construye la palabra ARINC 429 para un parametro multiplexado
92 static uint32_t buildScheduledWord(uint8_t schSlot, FlightData &flight,
    BaroData &baro, GPSData &gpsData, RadarData &radarData, ToFData &tofData,
    uint8_t seq) {
93  switch (schSlot) {

```



```

94     case SCH_PRES_ALT: {
95         uint8_t ssm = baro.ok ? A429_SSM_BNR_NORMAL : A429_SSM_BNR_NO_DATA;
96         float altFt = baro.ok ? baro.altitude_m * 3.28084f : 0.0f;
97         return ARINC429::buildBNR(A429_LABEL_PRES_ALT, 0, altFt,
A429_LSB_PRES_ALT_FT, ssm);
98     }
99     case SCH_VSPEED: {
100         uint8_t ssm = baro.ok ? A429_SSM_BNR_NORMAL : A429_SSM_BNR_NO_DATA;
101         float vsFpm = baro.ok ? baro.vspeed_mps * 196.85f : 0.0f;
102         return ARINC429::buildBNR(A429_LABEL_VERT_SPEED, 0, vsFpm,
A429_LSB_VSPEED_FPM, ssm);
103     }
104     case SCH_RADAR_DIST: {
105         uint8_t ssm = (radarData.ok && radarData.bestDist_cm > 0) ?
A429_SSM_BNR_NORMAL : A429_SSM_BNR_NO_DATA;
106         float dist_m = radarData.bestDist_cm / 100.0f;
107         return ARINC429::buildBNR(A429_LABEL_RADAR_DIST, 0, dist_m,
A429_LSB_RADAR_DIST_M, ssm);
108     }
109     case SCH_RADIO_ALT: {
110         uint8_t ssm = (tofData.ok && tofData.valid) ? A429_SSM_BNR_NORMAL :
A429_SSM_BNR_NO_DATA;
111         float ft = tofData.distance_mm * 0.00328084f;  // mm -> ft
112         return ARINC429::buildBNR(A429_LABEL_RADIO_ALT, 0, ft,
A429_LSB_RADIO_ALT_FT, ssm);
113     }
114     case SCH_GPS_ALT: {
115         uint8_t ssm = gpsData.fix ? A429_SSM_BNR_NORMAL : A429_SSM_BNR_NO_DATA;
116         float ft = gpsData.fix ? gpsData.altitude_m * 3.28084f : 0.0f;
117         return ARINC429::buildBNR(A429_LABEL_GPS_ALT, 0, ft, A429_LSB_GPS_ALT_FT,
ssm);
118     }
119     case SCH_LATITUDE: {
120         uint8_t ssm = gpsData.fix ? A429_SSM_BNR_NORMAL : A429_SSM_BNR_NO_DATA;
121         // Se transmite el desplazamiento respecto a la referencia local con
122         // LSB fino para tener resolucion metrica (ver SensorData.h)
123         float latOff = gpsData.fix ? (float)(gpsData.lat - GPS_LAT_REF_DEG) : 0.0
f;
124         return ARINC429::buildBNR(A429_LABEL_LATITUDE, 0, latOff,
A429_LSB_LATLON_DEG, ssm);
125     }
126     case SCH_LONGITUDE: {
127         uint8_t ssm = gpsData.fix ? A429_SSM_BNR_NORMAL : A429_SSM_BNR_NO_DATA;
128         float lonOff = gpsData.fix ? (float)(gpsData.lon - GPS_LON_REF_DEG) : 0.0

```

```

    f;
129     return ARINC429::buildBNR(A429_LABEL_LONGITUDE, 0, lonOff,
    A429_LSB_LATLON_DEG, ssm);
130 }
131 case SCH_STATUS: {
132     uint32_t bits = buildStatusDiscrete(gpsData, baro, radarData, tofData,
    seq);
133     return ARINC429::buildDiscrete(A429_LABEL_STATUS, 0, bits,
    A429_SSM_BNR_NORMAL);
134 }
135 default:
136     return A429_EMPTY_WORD;
137 }
138 }
139
140 // Empaqueta el estado actual y lo transmite.
141 bool sendTelemetry(FlightData &flight, BaroData &baro, GPSData &gpsData,
    RadarData &radarData, ToFData &tofData) {
142     ARINC429Packet pkt;
143     unsigned long now = millis();
144
145     // Slots fijos 0-2: actitud a 20 Hz (cada paquete los lleva)
146     // Convertimos heading a rango [-180, +180] para que la codificacion
147     // BNR signed sea simetrica.
148     float hdg = flight.heading;
149     if (hdg > 180.0f) hdg -= 360.0f;
150     pkt.words[0] = ARINC429::buildBNR(A429_LABEL_PITCH, 0, flight.pitch,
    A429_LSB_ANGLE_DEG, A429_SSM_BNR_NORMAL);
151     pkt.words[1] = ARINC429::buildBNR(A429_LABEL_ROLL, 0, flight.roll,
    A429_LSB_ANGLE_DEG, A429_SSM_BNR_NORMAL);
152     pkt.words[2] = ARINC429::buildBNR(A429_LABEL_HDG_MAG, 0, hdg,
    A429_LSB_ANGLE_DEG, A429_SSM_BNR_NORMAL);
153
154     // Slots multiplexados 3-7: planificador tipo "earliest deadline first".
155     // Para cada parametro se calcula cuanto tarde llega respecto a su
156     // periodo nominal, en cada slot se elige el mas atrasado de entre
157     // los que ya tocaba enviar
158     long lateness[SCH_COUNT];
159     bool ready[SCH_COUNT];
160
161     for (uint8_t i = 0; i < SCH_COUNT; i++) {
162         long elapsed = (long)(now - schedLastSent[i]);
163         lateness[i] = elapsed - schedPeriodMs[i];
164         ready[i] = (lateness[i] >= 0);

```

```

165     }
166
167     for (uint8_t slot = 3; slot < 8; slot++) {
168         int8_t bestIdx = -1;
169         long bestLate = -1;
170         for (uint8_t i = 0; i < SCH_COUNT; i++) {
171             if (ready[i] && lateness[i] > bestLate) {
172                 bestLate = lateness[i];
173                 bestIdx = (int8_t)i;
174             }
175         }
176
177         if (bestIdx < 0) {
178             // SI ningun parametro tocaba todavia se rellena con palabra vacia
179             // (label 0, SSM=NO_DATA) que el receptor ignora
180             pkt.words[slot] = A429_EMPTY_WORD;
181         } else {
182             // El status word se construye con el txSeqNumber actual
183             // El incremento se hace justo despues, solo cuando el slot elegido es el
184             // status, asi el receptor detecta status words perdidos por el salto de
185             // secuencia. El orden build-antes-de-incremento es lo que mantiene la
186             // numeracion: el paquete lleva N, el siguiente status llevara N+1.
187             pkt.words[slot] = buildScheduledWord((uint8_t)bestIdx, flight, baro,
188             gpsData, radarData, tofData, txSeqNumber);
189             schedLastSent[bestIdx] = now;
190             ready[bestIdx] = false;
191             if (bestIdx == SCH_STATUS) txSeqNumber++;
192         }
193
194         bool success = radio.write(&pkt, sizeof(pkt));
195         txCount++;
196         if (!success) txFailCount++;
197         return success;
198     }

```

Código 1.11: Transmisor NRF24L01 con payload ARINC 429

```

1  // ARINC429.h - codificacion/decodificacion de palabras ARINC 429.
2  //
3  // Implementa la capa de palabra (formato, etiquetas, BNR, SSM,
4  // paridad) del estandar ARINC 429 / Mark 33 DITS. La capa fisica

```

```

5 // diferencial bipolar NO sera implementada. Las palabras son de
6 // 32 bits y se pueden enviar por el transporte que se quiera (NRF24,
7 // UART, SPI, transceiver real como HI-3593...).
8 //
9 // Formato de palabra (32 bits):
10 // bits 0-7 label (etiqueta, 8 bits, octal)
11 // bits 8-9 SDI (Source/Destination Identifier)
12 // bits 10-28 data (19 bits utiles)
13 // bits 29-30 SSM (Sign/Status Matrix)
14 // bit 31 parity (paridad impar)
15 //
16 // Uso basico:
17 // #include <ARINC429.h>
18 //
19 // uint32_t w = ARINC429::buildBNR(A429_LABEL_PITCH, 0, 30.0f,
20 //                                A429_LSB_ANGLE_DEG,
21 //                                A429_SSM_BNR_NORMAL);
22 // ...
23 // uint8_t label, ssm;
24 // bool valid;
25 // float pitch = ARINC429::parseBNR(w, A429_LSB_ANGLE_DEG,
26 //                                label, ssm, valid);
27 //
28 // Referencias:
29 // ARINC Specification 429 Part 1-17 -- Functional Description,
30 // Electrical Interface, Label Assignments and Word Formats
31 // AIM "ARINC 429 Tutorial" v2.2, 2019 (BNR, SSM, parity)
32 // ARINC 429 Part 1, Attachment 2 -- tabla de parametros con sus
33 // escalas, ranges y tasas de refresco
34
35 #ifndef ARINC429_H
36 #define ARINC429_H
37
38 #include <Arduino.h>
39
40 // Etiquetas estandar (en octal). Lista parcial -- las que usa el TFG.
41 // La spec completa esta en ARINC 429 Part 1, Attachment 2.
42 #define A429_LABEL_PITCH 0324 // Pitch Angle - BNR +/-180 deg
43 #define A429_LABEL_ROLL 0325 // Roll Angle - BNR +/-180 deg
44 #define A429_LABEL_HDG_MAG 0320 // Magnetic Heading - BNR +/-180 deg
45 #define A429_LABEL_PRES_ALT 0204 // Pressure Altitude - BNR ft
46 #define A429_LABEL_VERT_SPEED 0365 // Vertical Speed - BNR ft/min
47 #define A429_LABEL_LATITUDE 0310 // Present Position Lat - BNR deg
48 #define A429_LABEL_LONGITUDE 0311 // Present Position Lon - BNR deg

```

```

49 #define A429_LABEL_GPS_ALT      0076  // GPS Altitude (MSL) - BNR ft
50 #define A429_LABEL_RADIO_ALT    0164  // Radio Altitude - BNR ft
51 #define A429_LABEL_RADAR_DIST    0166  // No estandar (libre). Distancia radar.
52 #define A429_LABEL_STATUS        0270  // No estandar. Status discrete custom.
53
54 // SSM para BNR (bits 29-30)
55 #define A429_SSM_BNR_FAILURE     0x0    // Failure Warning
56 #define A429_SSM_BNR_NO_DATA     0x1    // No Computed Data
57 #define A429_SSM_BNR_TEST        0x2    // Functional Test
58 #define A429_SSM_BNR_NORMAL      0x3    // Normal Operation
59
60 // Escalas BNR (unidades por LSB).
61 // Calculadas como rango_completo / 2^18 para 18 bits utiles + 1 de signo.
62 #define A429_LSB_ANGLE_DEG       (180.0f / 262144.0f) // 6.866e-4 deg/LSB
63 #define A429_LSB_PRES_ALT_FT     0.5f           // +/- 131072 ft
64 #define A429_LSB_GPS_ALT_FT      0.5f           // igual que pres alt
65 #define A429_LSB_VSPEED_FPM      0.125f          // +/- 32768 fpm
66 #define A429_LSB_RADIO_ALT_FT    0.0625f         // +/- 16384 ft
67 #define A429_LSB_RADAR_DIST_M    0.01f           // +/- 2621 m
68
69 // Palabra "vacía": label = 0, SSM = NO_DATA. Sirve como relleno de
70 // slots cuando el transmisor no tiene nada que mandar. El receptor la
71 // reconoce por label = 0 y la descarta.
72 #define A429_EMPTY_WORD          0x20000000UL
73
74 // Constructor / parser de palabras ARINC 429
75 class ARINC429 {
76 public:
77     // Construye una palabra BNR de 32 bits con paridad impar.
78     static uint32_t buildBNR(uint8_t label, uint8_t sdi,
79                             float value, float lsb, uint8_t ssm);
80
81     // Construye una palabra discrete (sin escala, 19 bits crudos).
82     static uint32_t buildDiscrete(uint8_t label, uint8_t sdi,
83                                   uint32_t data19, uint8_t ssm);
84
85     // Decodifica una palabra BNR. valid = paridad OK y SSM = NORMAL.
86     static float parseBNR(uint32_t word, float lsb,
87                           uint8_t &label_out, uint8_t &ssm_out, bool &valid);
88
89     // Decodifica una palabra discrete.
90     static uint32_t parseDiscrete(uint32_t word,
91                                   uint8_t &label_out, uint8_t &ssm_out,
92                                   bool &valid);

```

```

93
94  // Comprueba paridad impar (32 bits).
95  static bool checkParity(uint32_t word);
96
97 private:
98  static uint32_t pack(uint8_t label, uint8_t sdi,
99                      uint32_t data19, uint8_t ssm);
100  static uint8_t oddParityBit(uint32_t w);
101 };
102
103 #endif

```

Código 1.12: Biblioteca ARINC 429 (código propio): cabecera

```

1  // ARINC429.cpp - implementacion del builder/parser ARINC 429.
2
3  #include "ARINC429.h"
4
5  // Trunca un float al rango representable por un entero de 19 bits con
6 // signo (-2^18 .. 2^18 - 1) y devuelve la representacion en
7 // complemento a dos sobre 19 bits. Si esta fuera de rango se satura.
8  static uint32_t toData19Signed(long scaled) {
9      if (scaled > 262143) scaled = 262143; // 2^18 - 1
10     if (scaled < -262144) scaled = -262144; // -2^18
11     return ((uint32_t)scaled) & 0x7FFFF; // mascara 19 bits
12 }
13
14 uint32_t ARINC429::pack(uint8_t label, uint8_t sdi,
15                        uint32_t data19, uint8_t ssm) {
16     uint32_t w = 0;
17     w |= (uint32_t)label; // bits 0-7
18     w |= (uint32_t)(sdi & 0x03) << 8; // bits 8-9
19     w |= (uint32_t)(data19 & 0x7FFFF) << 10; // bits 10-28
20     w |= (uint32_t)(ssm & 0x03) << 29; // bits 29-30
21     return w;
22 }
23
24 // Paridad impar sobre los 32 bits via XOR-fold + lookup en mascara
25 // 0x6996. El lookup (0x6996 >> nibble) & 1 devuelve 1
26 // si el popcount del valor original (antes del fold) es IMPAR. Asi
27 // que para un dato cuyos bits de paridad acumulada (despues del fold)
28 // dan 0 (popcount par), hay que devolver 1 para que el total sea impar.

```

```

29 uint8_t ARINC429::oddParityBit(uint32_t w) {
30     uint32_t x = w;
31     x ^= x >> 16;
32     x ^= x >> 8;
33     x ^= x >> 4;
34     x &= 0xF;
35     uint8_t inputIsOdd = (0x6996 >> x) & 1;
36     // Si el dato ya tiene popcount impar, parity=0 deja el total impar.
37     // Si el dato tiene popcount par, parity=1 lo hace impar.
38     return (inputIsOdd == 1) ? 0 : 1;
39 }
40
41 uint32_t ARINC429::buildBNR(uint8_t label, uint8_t sdi,
42                             float value, float lsb, uint8_t ssm) {
43     // Escala a LSBs y redondea al entero mas cercano
44     long scaled = lroundf(value / lsb);
45     uint32_t data19 = toData19Signed(scaled);
46
47     uint32_t w = pack(label, sdi, data19, ssm);
48     uint8_t parity = oddParityBit(w);
49     return w | ((uint32_t)parity << 31);
50 }
51
52 uint32_t ARINC429::buildDiscrete(uint8_t label, uint8_t sdi,
53                                   uint32_t data19, uint8_t ssm) {
54     uint32_t w = pack(label, sdi, data19 & 0x7FFFF, ssm);
55     uint8_t parity = oddParityBit(w);
56     return w | ((uint32_t)parity << 31);
57 }
58
59 bool ARINC429::checkParity(uint32_t word) {
60     // La palabra ya incluye su bit de paridad. Si la paridad es correcta,
61     // el numero total de unos en los 32 bits sera impar (paridad impar).
62     // El truco 0x6996 con xor-fold devuelve 1 si el popcount es IMPAR,
63     // que es exactamente lo que queremos verificar.
64     uint32_t x = word;
65     x ^= x >> 16;
66     x ^= x >> 8;
67     x ^= x >> 4;
68     x &= 0xF;
69     uint8_t totalIsOdd = (0x6996 >> x) & 1;
70     return totalIsOdd == 1;
71 }
72

```

```

73 float ARINC429::parseBNR(uint32_t word, float lsb,
74                             uint8_t &label_out, uint8_t &ssm_out, bool &valid) {
75     label_out = (uint8_t)(word & 0xFF);
76     ssm_out    = (uint8_t)((word >> 29) & 0x03);
77     valid      = checkParity(word) && (ssm_out == A429_SSM_BNR_NORMAL);
78
79     // Extrae los 19 bits del campo de datos como entero con signo
80     uint32_t data19 = (word >> 10) & 0x7FFFF;
81     // Sign-extend a 32 bits: si el bit 18 esta a 1 el numero es negativo
82     int32_t signed19;
83     if (data19 & 0x40000) {
84         signed19 = (int32_t)(data19 | 0xFFF80000); // rellena con 1s
85     } else {
86         signed19 = (int32_t)data19;
87     }
88     return signed19 * lsb;
89 }
90
91 uint32_t ARINC429::parseDiscrete(uint32_t word,
92                                   uint8_t &label_out, uint8_t &ssm_out,
93                                   bool &valid) {
94     label_out = (uint8_t)(word & 0xFF);
95     ssm_out    = (uint8_t)((word >> 29) & 0x03);
96     valid      = checkParity(word) && (ssm_out == A429_SSM_BNR_NORMAL);
97     return (word >> 10) & 0x7FFFF;
98 }

```

Código 1.13: Biblioteca ARINC 429 (código propio): implementación

```

1  // FlightInstruments_Receiver.ino - Receptor de telemetria y puente WiFi
2
3  #include <SPI.h>
4  #include <nRF24L01.h>
5  #include <RF24.h>
6  #include <ARINC429.h>
7  #include "SensorData.h"
8  #include <WiFi.h>
9  #include <WiFiUdp.h>
10
11 // SPI del NRF24L01 sobre el ESP32-C3 Super Mini
12 #define NRF_SCK      4
13 #define NRF_MISO     5

```



```

14 #define NRF_MOSI      6
15 #define CE_PIN        3
16 #define CSN_PIN       7
17
18 // Radio. El canal y data rate tienen que coincidir con el transmisor
19 #define NRF_CHANNEL    108
20 #define NRF_DATA_RATE  RF24_250KBPS
21 #define NRF_PA_LEVEL   RF24_PA_HIGH
22 const byte addressRX[6] = "SENSR";
23
24 // Punto de acceso WiFi del receptor, para uso en android
25 // El movil con la apk se conecta a esta red y escucha el UDP broadcast
26 const char *ssid = "FlightData_Network";
27 const char *password = "12345678";
28 WiFiUDP udp;
29 const int udpPort = 12345;
30 IPAddress broadcastIP(192, 168, 4, 255);
31
32 #define PRINT_RATE_HZ      20
33 #define PRINT_INTERVAL_MS (1000 / PRINT_RATE_HZ)
34
35 // Estado decodificado de la telemetria. Funciona como cache de
36 // hold-last-good: cada label refresca su campo cuando llega una palabra
37 struct DecodedState {
38     float    pitch_deg;
39     float    roll_deg;
40     float    heading_deg;
41     float    pres_alt_ft;
42     float    vspeed_fpm;
43     double   lat_deg;
44     double   lon_deg;
45     float    gps_alt_ft;
46     float    radio_alt_ft;
47     float    radar_dist_m;
48     uint8_t  satellites;
49     uint8_t  status_flags;    // byte de sensores OK que va en $TEL
50     uint8_t  radar_status;   // byte empaquetado: zona + alerta + plataforma
51     uint8_t  last_seq;
52 };
53
54 DecodedState st = {0};
55
56 // Bits del byte de status que va en el campo 10 del paquete $TEL.
57 #define STATUS_IMU_OK      0x01

```

```
58 #define STATUS_BARO_OK    0x02
59 #define STATUS_GPS_FIX    0x04
60 #define STATUS_RADIO_OK   0x08
61 #define STATUS_RADAR_OK   0x10
62 #define STATUS_TOF_OK     0x20
63
64 RF24 radio(CE_PIN, CSN_PIN);
65 ARINC429Packet rxPkt;
66
67 uint32_t rxCount = 0;
68 uint32_t lostCount = 0;
69 uint32_t statusRxCount = 0; // Se mantiene la inicialización aquí
70 int32_t lastSeqRx = -1;
71 unsigned long lastRxTime = 0;
72 unsigned long lastPrintTime = 0;
73 unsigned long lastStatsTime = 0;
74
75 void setup() {
76     Serial.begin(115200);
77     delay(2000);
78
79     Serial.println("Flight Instruments Receiver & WiFi Bridge");
80
81     // WiFi en modo AP (Access Point)
82     WiFi.softAP(ssid, password);
83     udp.begin(udpPort);
84     Serial.print("WiFi AP OK. IP: ");
85     Serial.println(WiFi.softAPIP());
86
87     // NRF24L01: si falla aqui no hay nada que recibir, asi que se bloquea
88     // el arranque
89     SPI.begin(NRF_SCK, NRF_MISO, NRF_MOSI, -1);
90     if (!radio.begin(&SPI)) {
91         Serial.println("Error NRF24L01");
92         while (true) { delay(1000); }
93     }
94
95     radio.setPALevel(NRF_PA_LEVEL);
96     radio.setDataRate(NRF_DATA_RATE);
97     radio.setChannel(NRF_CHANNEL);
98     radio.setPayloadSize(sizeof(ARINC429Packet));
99     radio.openReadingPipe(1, addressRX);
100    radio.startListening();
101
```

```

102   Serial.println("Escuchando telemetria...");
103 }
104
105 // Decodifica una palabra ARINC 429 y actualiza el campo correspondiente
106 // de DecodedState
107 static void processWord(uint32_t word) {
108     uint8_t label = (uint8_t)(word & 0xFF);
109     uint8_t ssm    = (uint8_t)((word >> 29) & 0x03);
110
111     // Palabras vacías (label 0) y palabras corruptas se descartan
112     if (label == 0) return;
113     if (!ARINC429::checkParity(word)) return;
114
115     bool dataOk = (ssm == A429_SSM_BNR_NORMAL);
116     uint8_t lblOut, ssmOut;
117     bool valid;
118
119     switch (label) {
120         case A429_LABEL_PITCH:
121             if (dataOk) st.pitch_deg = ARINC429::parseBNR(word, A429_LSB_ANGLE_DEG,
122                 lblOut, ssmOut, valid);
123             break;
124         case A429_LABEL_ROLL:
125             if (dataOk) st.roll_deg = ARINC429::parseBNR(word, A429_LSB_ANGLE_DEG,
126                 lblOut, ssmOut, valid);
127             break;
128         case A429_LABEL_HDG_MAG:
129             if (dataOk) {
130                 // Heading codificado como signed [-180, +180]; el display espera [0,
131                 // 360]
132                 float h = ARINC429::parseBNR(word, A429_LSB_ANGLE_DEG, lblOut, ssmOut,
133                     valid);
134                 if (h < 0.0f) h += 360.0f;
135                 st.heading_deg = h;
136             }
137             break;
138         case A429_LABEL_PRES_ALT:
139             if (dataOk) st.pres_alt_ft = ARINC429::parseBNR(word,
140                 A429_LSB_PRES_ALT_FT, lblOut, ssmOut, valid);
141             break;
142         case A429_LABEL_VERT_SPEED:
143             if (dataOk) st.vspeed_fpm = ARINC429::parseBNR(word, A429_LSB_VSPEED_FPM,
144                 lblOut, ssmOut, valid);
145             break;
146     }
147 }

```

```

140     case A429_LABEL_LATITUDE:
141         if (dataOk) st.lat_deg = GPS_LAT_REF_DEG + ARINC429::parseBNR(word,
A429_LSB_LATLON_DEG, lblOut, ssmOut, valid);
142         break;
143     case A429_LABEL_LONGITUDE:
144         if (dataOk) st.lon_deg = GPS_LON_REF_DEG + ARINC429::parseBNR(word,
A429_LSB_LATLON_DEG, lblOut, ssmOut, valid);
145         break;
146     case A429_LABEL_GPS_ALT:
147         if (dataOk) st.gps_alt_ft = ARINC429::parseBNR(word, A429_LSB_GPS_ALT_FT,
lblOut, ssmOut, valid);
148         break;
149     case A429_LABEL_RADIO_ALT:
150         // Reset explicito a 0 si no hay datos nuevos
151         if (dataOk) st.radio_alt_ft = ARINC429::parseBNR(word,
A429_LSB_RADIO_ALT_FT, lblOut, ssmOut, valid);
152         else st.radio_alt_ft = 0;
153         break;
154     case A429_LABEL_RADAR_DIST:
155         if (dataOk) st.radar_dist_m = ARINC429::parseBNR(word,
A429_LSB_RADAR_DIST_M, lblOut, ssmOut, valid);
156         else st.radar_dist_m = 0;
157         break;
158     case A429_LABEL_STATUS: {
159         // Discrete: 19 bits utiles con flags, contador de satelites,
160         // zona del radar, alerta y numero de secuencia.
161         uint32_t bits = ARINC429::parseDiscrete(word, lblOut, ssmOut, valid);
162         if (!valid) break;
163
164         statusRxCount++; // Incremento tras parseo válido
165
166         st.satellites = (uint8_t)(bits & A429_STATUS_SATS_MASK);
167
168         // Traduce los bits del status word ARINC al layout del byte de
169         // status del $TEL. IMU y RADIO se ponen incondicionalmente
170         uint8_t flags = STATUS_IMU_OK | STATUS_RADIO_OK;
171         if (bits & A429_STATUS_FIX) flags |= STATUS_GPS_FIX;
172         if (bits & A429_STATUS_BARO_OK) flags |= STATUS_BARO_OK;
173         if (bits & A429_STATUS_RADAR_OK) flags |= STATUS_RADAR_OK;
174         if (bits & A429_STATUS_TOF_OK) flags |= STATUS_TOF_OK;
175         st.status_flags = flags;
176
177         // Empaqueta zona del radar (bits 0-1), alerta de target movil
178         // (bit 2) y plataforma en movimiento (bit 3) en un solo byte

```

```

179     // para el campo 14 del $TEL.
180     uint8_t a = (uint8_t)((bits & A429_STATUS_RADAR_ZONE_MASK) >>
A429_STATUS_RADAR_ZONE_SH);
181     if (bits & A429_STATUS_RADAR_ALERT) a |= 0x04;
182     if (bits & A429_STATUS_PLAT_MOVING) a |= 0x08;
183     st.radar_status = a;
184
185     // Deteccion de perdidas por gap en el numero de secuencia.
186     uint8_t curSeq = (uint8_t)((bits >> A429_STATUS_SEQ_SH) & 0x3F);
187     if (lastSeqRx >= 0) {
188         uint8_t expected = (uint8_t)((lastSeqRx + 1) & 0x3F);
189         if (curSeq != expected) {
190             uint8_t gap = (uint8_t)((curSeq - expected) & 0x3F);
191             if (gap < 32) lostCount += gap;
192         }
193     }
194     lastSeqRx = curSeq;
195     break;
196 }
197 }
198 }
199
200 void loop() {
201     unsigned long now = millis();
202
203     // Recepcion: cada paquete entra como 8 palabras ARINC y se procesan
204     // todas, aunque alguna sea EMPTY_WORD
205     if (radio.available()) {
206         radio.read(&rxPkt, sizeof(rxPkt));
207         rxCount++;
208         lastRxTime = now;
209
210         for (uint8_t i = 0; i < 8; i++) {
211             processWord(rxPkt.words[i]);
212         }
213     }
214
215     // Salida a 20 Hz: serializa el estado actual como sentencia CSV
216     // "$TEL,..." que el display Processing parsea como entrada.
217     if (now - lastPrintTime >= PRINT_INTERVAL_MS) {
218         lastPrintTime = now;
219         if (rxCount > 0) {
220             // Conversiones de unidades para el display
221             float alt_m      = st.pres_alt_ft / 3.28084f;

```

```

222     float vs_mps      = st.vspeed_fpm / 196.85f;
223     float gps_alt_m = st.gps_alt_ft / 3.28084f;
224
225     String telPacket = "$TEL," +
226                       String(st.pitch_deg, 1) + "," +
227                       String(st.roll_deg, 1) + "," +
228                       String(st.heading_deg, 1) + "," +
229                       String(alt_m, 1) + "," +
230                       String(vs_mps, 2) + "," +
231                       String(st.lat_deg, 6) + "," +
232                       String(st.lon_deg, 6) + "," +
233                       String((int)gps_alt_m) + "," +
234                       String(st.satellites) + "," +
235                       String(st.status_flags) + "," +
236                       String((int)(st.radar_dist_m * 100)) + "," +
237                       String((int)(st.radio_alt_ft * 304.8)) + ",0," +
238                       String(st.radar_status) + "*";
239
240     // Doble emision: serial USB para conexion local y UDP broadcast
241     Serial.println(telPacket);
242
243     udp.beginPacket(broadcastIP, udpPort);
244     udp.print(telPacket);
245     udp.endPacket();
246 }
247 }
248
249 // Telemetria del enlace a 1 Hz
250 if (now - lastStatsTime >= 1000) {
251     static uint32_t prevRxCount = 0;
252     static uint32_t prevLostCount = 0;
253
254     uint32_t rxThisSec = rxCount - prevRxCount;
255     uint32_t lostThisSec = lostCount - prevLostCount;
256
257     // Nueva lógica de cálculo para porcentaje de pérdidas basándose en
paquetes de status
258     uint32_t statusTotal = statusRxCount + lostCount;
259     float lossPercent = (statusTotal > 0) ? (lostCount * 100.0f / statusTotal)
260     : 0.0f;
261
262     Serial.print("$STATS,");
263     Serial.print(rxThisSec); Serial.print(",");
264     Serial.print(lostThisSec); Serial.print(",");

```

```
264     Serial.print(lossPercent, 1); Serial.print(",");
265     Serial.print(rxCount);
266     Serial.println("*");
267
268     prevRxCount = rxCount;
269     prevLostCount = lostCount;
270     lastStatsTime = now;
271
272     // Aviso de enlace caído: 3 s sin recibir nada
273     if (lastRxTime > 0 && (now - lastRxTime > 3000)) {
274         Serial.println("$WARN, LINK_LOST*");
275     }
276 }
277 }
```

Código 1.14: Receptor NRF24L01 y puente WiFi UDP (ESP32-C3)

Capítulo 2

ANEXO II. Código Processing

```
1 // Flight Instruments Display - archivo principal del PFD
2 // Lee la telemetria por puerto serie en formato "$TEL,...*" y pinta los
3 // instrumentos en layout Basic-T conforme a la guia FAA AC 25-11B. El
4 // archivo GPSMap.pde anade la vista de mapa, accesible con la tecla M
5
6 import processing.serial.*;
7
8 Serial myPort;
9
10 // Valores que se pintan en pantalla
11 float pitch = 0;           // grados (nariz arriba +)
12 float roll = 0;           // grados (ala derecha abajo +)
13 float heading = 0;        // 0-360 deg, magnetico
14 float slip = 0;          // deflexion de la bola de slip/skid
15 float altitude = 0;       // ft MSL
16 float verticalSpeed = 0;   // ft/min
17 float groundSpeed = 0;    // knots (desde GPS)
18
19 // Valores objetivo (ultimo paquete recibido)
20 float targetPitch = 0;
21 float targetRoll = 0;
22 float targetHeading = 0;
23 float targetAltitude = 0;
24 float targetVerticalSpeed = 0;
25 float targetGroundSpeed = 0;
26 float targetRadarDist = 0;
27 float targetTofDist = 0;
28 float radarDist = 0;
29 float tofDist = 0;
```



```
30
31 // Referencias que ajusta el piloto
32 float targetHdgBug = 0;    // heading bug (deg)
33 float baroSetting = 29.92; // inHg (ajuste Kollsman)
34
35 // Input
36 boolean keyLeft = false, keyRight = false;
37
38 // Flags y timers de UI
39 boolean showHelp = false;
40 long lastDataTime = 0;
41 boolean linkActive = false;
42
43 // Escalado
44 float baseWidth = 1280;
45 float baseHeight = 800;
46
47 // Timing
48 float dt = 1.0 / 60.0;
49
50 void settings() {
51     size(1280, 800, P2D); // Engine cambiado para mejor render
52     smooth(8);
53     pixelDensity(1);
54 }
55
56 void setup() {
57     surface.setResizable(true);
58     frameRate(60);
59
60     initGPSMap();
61
62     // Apertura del puerto serie
63     try {
64         String portName = "COM6";
65
66         myPort = new Serial(this, portName, 115200);
67         myPort.bufferUntil('\n'); // dispara serialEvent() una vez por linea
68         println("Escuchando telemetria en: " + portName);
69
70     } catch (Exception e) {
71         println(e.getMessage());
72         println("Puertos disponibles:");
73         for (String p : Serial.list()) {
```

```

74     println("-" + p);
75 }
76 }
77 }
78
79 void draw() {
80     background(30);
81
82     dt = 1.0 / frameRate;
83     if (dt > 0.05) dt = 0.05;
84
85     // El enlace se considera perdido tras 2 s sin recibir paquetes
86     if (millis() - lastDataTime > 2000) {
87         linkActive = false;
88     }
89
90     updateGPSData();
91
92     // gpsSpeed se actualiza en updateGPSData() a partir de la traza GPS
93     targetGroundSpeed = gpsSpeed / 1.852; // km/h -> knots
94
95     // Heading bug
96     float hdgBugRate = 45; // deg/s cuando se mantiene la tecla
97     if (keyLeft) targetHdgBug -= hdgBugRate * dt;
98     if (keyRight) targetHdgBug += hdgBugRate * dt;
99     if (targetHdgBug < 0) targetHdgBug += 360;
100    if (targetHdgBug >= 360) targetHdgBug -= 360;
101
102    // Suavizado por interpolacion lineal
103    // El valor 0.4 un buen balance entre respuesta y suavidad
104    // para la cadencia de 20 Hz del paquete.
105    if (linkActive) {
106        float smoothFactor = 0.4;
107        float diffpitch = targetPitch - pitch;
108        float diffroll = targetRoll - roll;
109
110        if (abs(diffpitch)>0.1){
111            pitch = lerp(pitch, targetPitch, smoothFactor);
112        }
113        if (abs(diffroll)>0.1){
114            roll = lerp(roll, targetRoll, smoothFactor);
115        }
116        altitude = lerp(altitude, targetAltitude, smoothFactor);
117        verticalSpeed = lerp(verticalSpeed, targetVerticalSpeed, smoothFactor);

```

```
118     radarDist = targetRadarDist;
119     tofDist = targetTofDist;
120     groundSpeed = lerp(groundSpeed, targetGroundSpeed, smoothFactor);
121
122     // El heading requiere aritmetica circular para no atravesar el rango
123     // largo al cruzar la discontinuidad 0/360
124     float diff = targetHeading - heading;
125     if (diff > 180) diff -= 360;
126     if (diff < -180) diff += 360;
127     if (abs(diff) > 0.3) {
128         heading += diff * smoothFactor;
129     }
130     if (heading >= 360) heading -= 360;
131     if (heading < 0) heading += 360;
132 }
133
134 // Escalado del canvas
135 float scaleFactor = height / baseHeight;
136 pushMatrix();
137 scale(scaleFactor);
138
139 float virtualWidth = width / scaleFactor;
140 float centeringOffset = (virtualWidth - baseWidth) / 2;
141 translate(centeringOffset, 0);
142
143 // Render
144 if (showMap) {
145     drawGPSMap();
146 } else {
147     float cx = baseWidth / 2;
148     float cy = baseHeight / 2 - 50;
149
150     // Basic-T
151     drawAttitudeIndicator(cx, cy, 400);
152     drawGroundSpeedIndicator(cx - 250, cy, 80, 400);
153     drawAltimeter(cx + 250, cy, 80, 400, altitude);
154     drawVSI(cx + 330, cy, 50, 400);
155     drawHeadingIndicator(cx, 630, 550, 50);
156     drawProximityIndicator(cx + 345, 630, 110, 60);
157
158     drawStatusBar();
159     if (showHelp) drawHelpOverlay();
160 }
161
```

```
162     popMatrix();
163 }
164
165 // Parser del paquete "$TEL,...*"
166 // Formato de 15 campos:
167 //   $TEL,pitch,roll,hdg,alt,vspeed,lat,lon,gpsAlt,sats,flags,
168 //       radarDist,tofDist,radarEnergy,radarAlert*
169 void serialEvent(Serial p) {
170     try {
171         String inString = p.readStringUntil('\n');
172
173         if (inString != null) {
174             inString = trim(inString);
175
176             if (inString.startsWith("$TEL,")) {
177
178                 // Eliminacion del asterisco terminador
179                 if (inString.endsWith("*")) {
180                     inString = inString.substring(0, inString.length() - 1);
181                 }
182
183                 String[] data = split(inString, ',');
184
185                 if (data.length >= 15) {
186                     // Se actualizan los valores objetivo, no los de display
187                     targetPitch = float(data[1]);
188                     targetRoll = float(data[2]);
189                     targetHeading = float(data[3]);
190
191                     targetAltitude = float(data[4]) * 3.28084;           // m -> ft
192                     targetVerticalSpeed = float(data[5]) * 196.8504;    // m/s -> ft/min
193
194                     // targetGroundSpeed se calcula en draw() a partir de gpsSpeed
195
196                     gpsLat = float(data[6]);
197                     gpsLon = float(data[7]);
198                     gpsSats = int(data[9]);
199                     int statusFlags = int(data[10]);
200                     gpsFix = (statusFlags & 0x04) != 0;
201
202                     targetRadarDist = float(data[11]);
203                     targetTofDist = float(data[12]);
204
205                     linkActive = true;
```

```
206         lastDataTime = millis();
207     }
208 }
209 }
210 } catch (Exception e) {
211     // Paquete corrupto: causado por posibles interferencias RF en el
212     // enlace inalámbrico o por bytes perdidos en el puerto USB
213     println("Paquete corrupto, ignorado: " + e.getMessage());
214 }
215 }
216
217 // Teclado
218 void keyPressed() {
219     if (key == 'm' || key == 'M') {
220         showMap = !showMap;
221     }
222     // En vista de mapa, las teclas de zoom se gestionan en GPSMap.pde
223     if (showMap) {
224         handleMapKeys();
225         return;
226     }
227     if (key == 'h' || key == 'H') showHelp = !showHelp;
228
229     // Ajuste de la presion de referencia del barometro
230     if (key == ']') baroSetting = constrain(baroSetting + 0.01, 28.00, 31.50);
231     if (key == '[') baroSetting = constrain(baroSetting - 0.01, 28.00, 31.50);
232
233     if (key == CODED) {
234         if (keyCode == LEFT) keyLeft = true;
235         if (keyCode == RIGHT) keyRight = true;
236     }
237 }
238
239 void keyReleased() {
240     if (key == CODED) {
241         if (keyCode == LEFT) keyLeft = false;
242         if (keyCode == RIGHT) keyRight = false;
243     }
244 }
245
246 // Barra de estado arriba a la izquierda
247 void drawStatusBar() {
248     float y = 18;
249     textSize(13);
```

```

250   textAlign(LEFT, BASELINE);
251
252   if (linkActive) {
253       fill(100, 255, 100);
254       text("LIVE TELEMETRY (20Hz)", 15, y);
255   } else {
256       fill(255, 50, 50);
257       text("NO DATA / LINK LOST", 15, y);
258   }
259
260   y += 18;
261   fill(150);
262   text("P:" + nf(pitch, 0, 1) + "\u00B0 R:" + nf(roll, 0, 1) + "\u00B0 H:" +
        nf(heading, 0, 0) + "\u00B0", 15, y);
263   y += 16;
264   text("GS:" + int(groundSpeed) + "kt ALT:" + int(altitude) + "ft VS:" + int(
        verticalSpeed) + "fpm", 15, y);
265   y += 16;
266   fill(130);
267   text("BUG:" + nf(int(targetHdgBug), 3) + "\u00B0 BARO:" + nf(baroSetting, 2,
        2), 15, y);
268 }
269
270 // Panel de ayuda arriba a la derecha
271 void drawHelpOverlay() {
272     float ox = baseWidth - 260;
273     float oy = 15;
274     float lh = 16;
275
276     float panelH = 130;
277     fill(0, 160);
278     noStroke();
279     rectMode(CORNER);
280     rect(ox - 10, oy - 5, 255, panelH, 6);
281
282     fill(220);
283     textSize(12);
284     textAlign(LEFT, TOP);
285     text("[ H ]      Muestra/oculta esta ayuda", ox, oy); oy += lh;
286     text("[ M ]      Vista de mapa GPS", ox, oy); oy += lh;
287     text("[ [ / ] ]   Baro -/+", ox, oy); oy += lh;
288     text("[ \u2190 / \u2192 ]   Heading bug izq/der", ox, oy); oy += lh * 1.5;
289
290     fill(100, 255, 100);

```

```

291  text("DATA STREAM", ox, oy); oy += lh;
292  fill(180);
293  text("Esperando paquetes NRF24L01...", ox, oy);
294  }

```

Código 2.1: Código principal del visualizador de vuelo (Flight Display)

```

1  // Altimetro (altitud MSL en pies)
2  // Cinta vertical analoga a la del ASI con su propia escala y una ventana
3  // Kohlsman (ajuste barometrico) debajo, controlada por las teclas [ y ].
4  // El rango visible esta reducido respecto al estandar aeronautico para
5  // adaptar el instrumento a la operacion en banco de pruebas (que se pueda ver
   movimiento)
6
7  void drawAltimeter(float centerX, float centerY, float tapeWidth, float
   tapeHeight, float altitude) {
8      float halfH = tapeHeight / 2;
9      float halfW = tapeWidth / 2;
10     float rangeVisible = 100;           // ft visibles en la cinta
11     float pxPerFt = tapeHeight / rangeVisible;
12
13     // Fondo
14     fill(40);
15     stroke(80);
16     strokeWeight(2);
17     rectMode(CENTER);
18     rect(centerX, centerY, tapeWidth, tapeHeight);
19
20     // Marcas: cada 5 ft tick pequeno, cada 25 ft tick grande etiquetado
21     float minAlt = altitude - (rangeVisible / 2) - 10;
22     float maxAlt = altitude + (rangeVisible / 2) + 10;
23     int startTick = ceil(minAlt / 5.0) * 5;
24
25     for (int altVal = startTick; altVal <= maxAlt; altVal += 5) {
26         float delta = altVal - altitude;
27         float y = centerY - (delta * pxPerFt);
28
29         if (y < centerY - halfH + 2 || y > centerY + halfH - 2) continue;
30
31         float xLeft = centerX - halfW;
32         float tickLen = (altVal % 25 == 0) ? 15 : 8;
33         float weight = (altVal % 25 == 0) ? 2 : 1;

```

```
34     boolean showText = (altVal % 25 == 0);
35
36     stroke(255);
37     strokeWeight(weight);
38     line(xLeft, y, xLeft + tickLen, y);
39
40     if (showText) {
41         fill(255);
42         textAlign(LEFT, CENTER);
43         textSize(15);
44         text(str(altVal), xLeft + tickLen + 4, y - 1);
45     }
46 }
47
48 // Marca central (flecha apuntando a la izquierda)
49 float boxW = 60;
50 float boxH = 16;
51 float boxX = centerX - halfW + 15;
52
53 fill(0);
54 stroke(255);
55 strokeWeight(2);
56 beginShape();
57 vertex(boxX, centerY - boxH);
58 vertex(boxX + boxW, centerY - boxH);
59 vertex(boxX + boxW, centerY + boxH);
60 vertex(boxX, centerY + boxH);
61 vertex(boxX, centerY + 6);
62 vertex(boxX - 6, centerY);    // flecha hacia la izquierda
63 vertex(boxX, centerY - 6);
64 endShape(CLOSE);
65
66 fill(0, 255, 0);
67 textAlign(CENTER, CENTER);
68 textSize(18);
69 text(int(altitude), boxX + boxW / 2, centerY - 1);
70
71 // Etiqueta "ALT"
72 fill(180);
73 textAlign(CENTER, TOP);
74 textSize(11);
75 text("ALT", centerX, centerY - halfH + 4);
76
77 // Ventana Kohlsman: muestra el ajuste barometrico actual en inHg.
```



```

78  // Referencia estandar ISA: 29.92 inHg.
79  float baroY = centerY + halfH + 16;
80  fill(0);
81  stroke(80);
82  strokeWeight(1);
83  rectMode(CENTER);
84  rect(centerX, baroY, tapeWidth + 6, 22);
85
86  fill(0, 255, 255);
87  textAlign(CENTER, CENTER);
88  textSize(12);
89  text(nf(baroSetting, 2, 2) + " IN", centerX, baroY - 1);
90
91  rectMode(CORNER);
92  textAlign(LEFT);
93  }

```

Código 2.2: Código del indicador de altitud sobre el nivel del mar (Altimeter MSL)

```

1  // Indicador de actitud (horizonte artificial)
2  // El cielo y la tierra se pintan como dos rectangulos rotados segun
3  // los angulos pitch/roll del paquete de telemetria, sobre los que se
4  // superponen la escala de pitch y el arco de bank. Por encima se dibuja
5  // un rectangulo con un agujero circular que actua como mascara y solo
6  // deja visible la parte interior del instrumento
7
8  void drawAttitudeIndicator(float centerX, float centerY, float size) {
9      float radius = size / 2;
10     float pixelsPerDegree = size / 45;
11     float pitchOffset = pitch * pixelsPerDegree;
12
13     pushMatrix();
14     translate(centerX, centerY);
15     rotate(radians(-roll));
16
17     // Cielo: gradiente de azul oscuro a azul claro en 8 franjas
18     float coverage = size * 5;
19     int gradSteps = 8;
20     float stripH = (size * 3) / gradSteps;
21
22     for (int i = 0; i < gradSteps; i++) {
23         float t = (float) i / (gradSteps - 1);

```

```
24     fill(lerpColor(color(0, 40, 100), color(0, 140, 230), t));
25     noStroke();
26     rect(-coverage, -size * 3 + pitchOffset + i * stripH, coverage * 2, stripH
27     + 1);
28 }
29 // Tierra: marron claro a marron oscuro
30 for (int i = 0; i < gradSteps; i++) {
31     float t = (float) i / (gradSteps - 1);
32     fill(lerpColor(color(130, 85, 40), color(45, 25, 8), t));
33     noStroke();
34     rect(-coverage, pitchOffset + i * stripH, coverage * 2, stripH + 1);
35 }
36
37 // Linea del horizonte
38 stroke(255);
39 strokeWeight(2);
40 line(-size, pitchOffset, size, pitchOffset);
41
42 // Escala de pitch: marcas cada 5deg, numeradas cada 10deg
43 textAlign(CENTER, CENTER);
44 textSize(14);
45 int minDeg = floor((pitch - 30) / 5.0) * 5;
46 int maxDeg = ceil((pitch + 30) / 5.0) * 5;
47
48 for (int deg = minDeg; deg <= maxDeg; deg += 5) {
49     if (deg == 0) continue;
50
51     float y = pitchOffset - (deg * pixelsPerDegree);
52     float lineLength = (deg % 10 == 0) ? 60 : 30;
53
54     stroke(255);
55     strokeWeight(2);
56     line(-lineLength, y, lineLength, y);
57
58     if (deg % 10 == 0) {
59         fill(255);
60         noStroke();
61         text(abs(deg), -lineLength - 20, y);
62         text(abs(deg), lineLength + 20, y);
63     }
64 }
65
66 popMatrix();
```

```
67
68 // Arco de bank angle con marcas
69 float arcRadius = radius - 20;
70 stroke(255);
71 strokeWeight(2);
72 noFill();
73 arc(centerX, centerY, arcRadius * 2, arcRadius * 2, radians(-150), radians
    (-30));
74
75 int[] majorAngles = {30, 60};
76 int[] minorAngles = {10, 20, 45};
77
78 for (int angle : majorAngles) {
79     drawBankTick(centerX, centerY, arcRadius, angle, 15);
80     drawBankTick(centerX, centerY, arcRadius, -angle, 15);
81 }
82 for (int angle : minorAngles) {
83     drawBankTick(centerX, centerY, arcRadius, angle, 8);
84     drawBankTick(centerX, centerY, arcRadius, -angle, 8);
85 }
86 drawBankTick(centerX, centerY, arcRadius, 0, 15);
87
88 // Triangulo de referencia, fijo
89 fill(255);
90 noStroke();
91 float refY = centerY - arcRadius;
92 triangle(centerX - 8, refY - 12, centerX + 8, refY - 12, centerX, refY);
93
94 // Puntero de bank y bola de slip/skid
95 pushMatrix();
96 translate(centerX, centerY);
97 rotate(radians(-roll));
98
99 fill(255);
100 noStroke();
101 triangle(-8, -arcRadius + 12, 8, -arcRadius + 12, 0, -arcRadius);
102
103 // Bola de deslizamiento entre las dos referencias
104 float brickY = -radius + 40;
105 stroke(255);
106 strokeWeight(2);
107 line(-12, brickY - 6, -12, brickY + 6);
108 line(12, brickY - 6, 12, brickY + 6);
109
```

```
110 fill(255);
111 noStroke();
112 rectMode(CENTER);
113 rect(slip, brickY, 20, 8);
114 rectMode(CORNER);
115
116 popMatrix();
117
118 // Simbolo del avion, fijo y centrado
119 stroke(255, 200, 0);
120 strokeWeight(3);
121 line(centerX - 80, centerY, centerX - 30, centerY);
122 line(centerX + 30, centerY, centerX + 80, centerY);
123 fill(255, 200, 0);
124 noStroke();
125 ellipse(centerX, centerY, 10, 10);
126
127 // Mascara circular: rectangulo grande con un agujero redondo
128 // recortado. Todo lo pintado fuera del circulo queda tapado
129 float maskExt = size * 5;
130 fill(30);
131 noStroke();
132 beginShape();
133 vertex(centerX - maskExt, centerY - maskExt);
134 vertex(centerX + maskExt, centerY - maskExt);
135 vertex(centerX + maskExt, centerY + maskExt);
136 vertex(centerX - maskExt, centerY + maskExt);
137 beginContour();
138 // Paso de 2deg: 180 vertices, resolucion suficiente para que el
139 // contorno del circulo se vea suave sin penalizar el rendimiento
140 for (float a = 360; a >= 0; a -= 2) {
141     float angle = radians(a);
142     vertex(centerX + cos(angle) * radius, centerY + sin(angle) * radius);
143 }
144 endContour();
145 endShape(CLOSE);
146
147 // Bezel
148 stroke(80);
149 strokeWeight(4);
150 noFill();
151 ellipse(centerX, centerY, size, size);
152 }
153
```

```

154 void drawBankTick(float cx, float cy, float radius, int angle, float length) {
155     float screenAngle = radians(-90 - angle);
156     float x1 = cx + cos(screenAngle) * radius;
157     float y1 = cy + sin(screenAngle) * radius;
158     float x2 = cx + cos(screenAngle) * (radius - length);
159     float y2 = cy + sin(screenAngle) * (radius - length);
160
161     stroke(255);
162     strokeWeight(2);
163     line(x1, y1, x2, y2);
164 }

```

Código 2.3: Código del indicador de actitud / horizonte artificial (Attitude Indicator)

```

1  // Vista de mapa GPS - tiles topograficos del ICGC descargados en disco
2  //
3  // Referencias:
4  //   ICGC Base Map Service (capa topografica).
5  //   Licencia: CC-BY (Cataluna) / ODbL (resto del mundo).
6  //   https://www.icgc.cat/en/Geoinformation-and-Maps/Base-Map-Service
7  //   OpenStreetMap Wiki - "Slippy map tilenames"
8  //   FAA HF-STD-001B sec.5.6.3.2 - guia de map displays
9  //   FAA AC 25-11B sec.6.2.5 - ubicacion de "other information"
10
11 import java.util.HashMap;
12 import java.io.File;
13
14 // Datos GPS
15 float gpsLat = 0;
16 float gpsLon = 0;
17 float gpsSpeed = 0;
18 int   gpsSats = 0;
19 boolean gpsFix = false;
20
21 // Traza
22 // Los puntos son float[2] {lat, lon}. Un null en la lista marca un "hueco"
23 // para que drawMapTrail no una con una linea recta dos posiciones separadas en
   el tiempo
24 ArrayList<float[]> gpsTrail;
25 int maxTrailPoints = 2000;
26 long lastTrailTime = 0;
27 int trailIntervalMs = 500;

```

```
28 boolean lastFixState = false; // para detectar transicion fix->sin fix
29
30 // Estado del mapa
31 boolean showMap = false;
32 int mapZoom = 15; // zoom entero Web Mercator
33 final int MAP_ZOOM_MIN = 8;
34 final int MAP_ZOOM_MAX = 17;
35 boolean mapCenterLocked = true; // true = el mapa sigue al avion
36 float mapCenterLat;
37 float mapCenterLon;
38
39 // Vista por defecto cuando aun no hay fix.
40 // Si despues llega, el mapa se recentra solo
41 final float DEFAULT_MAP_LAT = 41.5638;
42 final float DEFAULT_MAP_LON = 2.0222;
43
44 // Cache de tiles
45 HashMap<String, PImage> tileCache;
46 final int CACHE_MAX_SIZE = 300;
47
48 // Colores (paleta para mapa topografico claro)
49 final color MAP_FALLBACK_BG = color(240, 238, 232);
50 final color MAP_FALLBACK_GRID = color(200, 198, 190);
51 final color MAP_TRAIL = color(220, 30, 30);
52 final color MAP_TRAIL_OLD = color(220, 150, 150);
53 final color MAP_AIRCRAFT = color(0, 110, 220);
54 final color MAP_AIRCRAFT_LINE = color(255);
55 final color MAP_WARNING = color(220, 140, 0);
56 final color MAP_PANEL_BG = color(255, 220);
57 final color MAP_PANEL_TEXT = color(30);
58 final color MAP_STRIP_BG = color(30);
59 final color MAP_STRIP_LABEL = color(180);
60 final color MAP_STRIP_VALUE = color(230);
61
62 // Llamar desde setup()
63 void initGPSMap() {
64     gpsTrail = new ArrayList<float[]>();
65     tileCache = new HashMap<String, PImage>();
66     mapCenterLat = DEFAULT_MAP_LAT;
67     mapCenterLon = DEFAULT_MAP_LON;
68     // Los tiles que falten se dibujan como celda gris
69 }
70
71 // Llamar desde draw() antes del render
```

```

72 void updateGPSData() {
73     // Deteccion de la transicion fix -> sin fix. Cuando se pierde el fix
74     // se inserta un null en la traza para que, al recuperar la senal,
75     // drawMapTrail no una los dos extremos con una recta
76     if (lastFixState && !gpsFix && gpsTrail.size() > 0) {
77         // Solo se inserta el marcador si el ultimo punto no es null
78         if (gpsTrail.get(gpsTrail.size() - 1) != null) {
79             gpsTrail.add(null);
80         }
81     }
82     lastFixState = gpsFix;
83
84     // Anade un punto a la traza cada trailIntervalMs solo si hay fix
85     if (gpsFix && millis() - lastTrailTime > trailIntervalMs) {
86         // Calculo de la velocidad sobre el suelo (GS) entre el ultimo punto
87         // valido y la nueva posicion, mediante proyeccion: dx = dLon*cos(lat), dy
88         // = dLat.
89
90         float[] lastPt = null;
91         long lastDt = 0;
92         for (int i = gpsTrail.size() - 1; i >= 0; i--) {
93             if (gpsTrail.get(i) != null) {
94                 lastPt = gpsTrail.get(i);
95                 lastDt = millis() - lastTrailTime;
96                 break;
97             }
98         }
99
100         if (lastPt != null && lastDt > 0 && lastDt < 3000) {
101             float dLat = gpsLat - lastPt[0];
102             float dLon = (gpsLon - lastPt[1]) * cos(radians(gpsLat));
103             float dist_m = sqrt(dLat * dLat + dLon * dLon) * 111320.0;
104             float speed_ms = dist_m / (lastDt / 1000.0);
105             float speed_kmh = speed_ms * 3.6;
106
107             // Filtro EMA sobre la velocidad calculada para absorber el jitter
108             // de la posicion GPS.
109             float alpha = 0.4;
110             gpsSpeed = gpsSpeed * (1 - alpha) + speed_kmh * alpha;
111         }
112
113         gpsTrail.add(new float[]{gpsLat, gpsLon});
114         if (gpsTrail.size() > maxTrailPoints) {
115             gpsTrail.remove(0);

```

```
115     }
116     lastTrailTime = millis();
117 }
118
119 // Sin fix, el GS decae suavemente hacia cero en lugar de quedarse
120 // congelado en el ultimo valor calculado
121 if (!gpsFix) {
122     gpsSpeed *= 0.9;
123     if (gpsSpeed < 0.1) gpsSpeed = 0;
124 }
125
126 // Auto-centrado del mapa sobre el avion mientras el lock este activo
127 if (mapCenterLocked && gpsFix) {
128     mapCenterLat = gpsLat;
129     mapCenterLon = gpsLon;
130 }
131 }
132
133 // Proyeccion Web Mercator
134
135 float lonToPixelX(float lon, int z) {
136     return (lon + 180.0) / 360.0 * 256.0 * pow(2, z);
137 }
138
139 float latToPixelY(float lat, int z) {
140     float latRad = radians(lat);
141     return (1 - log(tan(latRad) + 1.0 / cos(latRad)) / PI)
142           / 2.0 * 256.0 * pow(2, z);
143 }
144
145 // Carga de tiles: lectura desde disco, con cache en RAM
146 // Devuelve null para tiles que no existen en disco (drawTiles pinta
147 // una celda gris en esos huecos). El null se cachea igual que un tile
148 // valido, asi no se llama a File.exists() en bucle
149 PImage getTile(int z, int x, int y) {
150     String key = z + "/" + x + "/" + y;
151
152     if (tileCache.containsKey(key)) {
153         return tileCache.get(key);
154     }
155
156     String relativeBasePath = "../..//Maps/data/tiles/";
157     String tilePath = relativeBasePath + z + "/" + x + "/" + y + ".png";
158     String fullPath = sketchPath(tilePath);
```



```
159   PImage img = null;
160
161   // El tile se valida antes de loadImage() para evitar mensajes de
162   // "can't find file" en la consola cuando la coordenada cae fuera del
163   // set descargado.
164   File f = new File(fullPath);
165   if (f.exists()) {
166       img = loadImage(fullPath);
167       if (img != null && img.width == 0) img = null;
168   }
169
170   // Cache: cuando se llena, se vacia por completo.
171   if (tileCache.size() > CACHE_MAX_SIZE) {
172       tileCache.clear();
173   }
174
175   tileCache.put(key, img);
176   return img;
177 }
178
179 // Dibujo del mapa
180 void drawGPSMap() {
181     float mapX = 0;
182     float mapY = 0;
183     float mapW = baseWidth;
184     float mapH = baseHeight - 60; // franja inferior para el status
185     float mapCX = mapX + mapW / 2;
186     float mapCY = mapY + mapH / 2;
187
188     // Fondo, se ve por los huecos donde falten tiles
189     fill(MAP_FALLBACK_BG);
190     noStroke();
191     rect(mapX, mapY, mapW, mapH);
192
193     drawTiles(mapCX, mapCY, mapW, mapH);
194     drawMapTrail(mapCX, mapCY);
195
196     if (gpsFix) {
197         drawAircraftSymbol(mapCX, mapCY);
198     }
199
200     drawCompassRose(mapW - 60, 60);
201     drawScaleBar(mapX + 20, mapH - 20);
202     drawZoomIndicator(mapX + 15, mapY + 20);
```

```

203 drawAttribution(mapW, mapH);
204
205 if (!gpsFix) {
206     drawNoFixWarning(mapCX, mapCY);
207 }
208
209 drawMapStatusStrip(mapX, mapH, mapW);
210 }
211
212 // Pinta la rejilla de tiles alrededor del centro del mapa
213 void drawTiles(float mapCX, float mapCY, float mapW, float mapH) {
214     int z = mapZoom;
215
216     float centerPxX = lonToPixelX(mapCenterLon, z);
217     float centerPxY = latToPixelY(mapCenterLat, z);
218
219     int centerTileX = (int)(centerPxX / 256);
220     int centerTileY = (int)(centerPxY / 256);
221
222     int halfX = (int)ceil(mapW / 512.0) + 1;
223     int halfY = (int)ceil(mapH / 512.0) + 1;
224
225     int maxTile = (int)pow(2, z);
226
227     imageMode(CORNER);
228     noTint();
229
230     for (int dy = -halfY; dy <= halfY; dy++) {
231         for (int dx = -halfX; dx <= halfX; dx++) {
232             int tx = centerTileX + dx;
233             int ty = centerTileY + dy;
234
235             // Coordenadas fuera del mundo Mercator -> skip
236             if (tx < 0 || tx >= maxTile || ty < 0 || ty >= maxTile) continue;
237
238             float screenX = mapCX + (tx * 256 - centerPxX);
239             float screenY = mapCY + (ty * 256 - centerPxY);
240
241             PImage tile = getTile(z, tx, ty);
242             if (tile != null) {
243                 image(tile, screenX, screenY, 256, 256);
244             } else {
245                 fill(MAP_FALLBACK_BG);
246                 stroke(MAP_FALLBACK_GRID);

```

```

247         strokeWeight(0.5);
248         rect(screenX, screenY, 256, 256);
249     }
250 }
251 }
252 }
253
254 // Traza GPS como polilinea, con fade a rojo hacia el punto mas reciente.
255 // Los puntos null marcan huecos (fix perdido): la linea se corta ahi
256 // y se retoma cuando vuelve a haber datos.
257 void drawMapTrail(float mapCX, float mapCY) {
258     if (gpsTrail.size() < 2) return;
259
260     int z = mapZoom;
261     float centerPxX = lonToPixelX(mapCenterLon, z);
262     float centerPxY = latToPixelY(mapCenterLat, z);
263     int total = gpsTrail.size();
264
265     noFill();
266     strokeWeight(2.5);
267
268     for (int i = 1; i < total; i++) {
269         float[] p0 = gpsTrail.get(i - 1);
270         float[] p1 = gpsTrail.get(i);
271
272         if (p0 == null || p1 == null) continue;
273
274         float x0 = mapCX + (lonToPixelX(p0[1], z) - centerPxX);
275         float y0 = mapCY + (latToPixelY(p0[0], z) - centerPxY);
276         float x1 = mapCX + (lonToPixelX(p1[1], z) - centerPxX);
277         float y1 = mapCY + (latToPixelY(p1[0], z) - centerPxY);
278
279         // age: 0 al principio de la traza, 1 al final
280         float age = map(i, 0, total - 1, 0, 1);
281         stroke(lerpColor(MAP_TRAIL_OLD, MAP_TRAIL, age));
282         line(x0, y0, x1, y1);
283     }
284 }
285
286 // Triangulo del avion, rotado al heading actual
287 void drawAircraftSymbol(float cx, float cy) {
288     pushMatrix();
289     translate(cx, cy);
290     rotate(radians(heading));

```

```
291
292   stroke(MAP_AIRCRAFT, 160);
293   strokeWeight(2);
294   line(0, -18, 0, -42);
295
296   // Forma del avion
297   stroke(MAP_AIRCRAFT_LINE);
298   strokeWeight(2);
299   fill(MAP_AIRCRAFT);
300   beginShape();
301   vertex(0, -14);
302   vertex(-9, 10);
303   vertex(0, 5);
304   vertex(9, 10);
305   endShape(CLOSE);
306
307   // Punto central
308   fill(MAP_AIRCRAFT_LINE);
309   noStroke();
310   ellipse(0, 0, 4, 4);
311
312   popMatrix();
313 }
314
315 // Rosa de los vientos arriba a la derecha
316 void drawCompassRose(float x, float y) {
317   float r = 30;
318
319   fill(MAP_PANEL_BG);
320   stroke(80);
321   strokeWeight(1);
322   ellipse(x, y, r * 2, r * 2);
323
324   textSize(11);
325   textAlign(CENTER, CENTER);
326
327   String[] labels = {"N", "E", "S", "W"};
328   for (int i = 0; i < 4; i++) {
329     float angle = i * HALF_PI - HALF_PI;
330     float tx = x + cos(angle) * (r - 10);
331     float ty = y + sin(angle) * (r - 10);
332     fill(i == 0 ? color(200, 30, 30) : color(60)); // N en rojo
333     noStroke();
334     text(labels[i], tx, ty);
```

```
335     }
336
337     // Aguja rotando con el heading
338     pushMatrix();
339     translate(x, y);
340     rotate(radians(heading));
341     stroke(MAP_AIRCRAFT);
342     strokeWeight(2);
343     line(0, 5, 0, -r + 8);
344     fill(MAP_AIRCRAFT);
345     noStroke();
346     triangle(0, -r + 5, -4, -r + 12, 4, -r + 12);
347     popMatrix();
348 }
349
350 // Barra de escala: dibuja la longitud equivalente en metros segun la
351 // resolucion real (m/px) de Web Mercator a la latitud actual.
352 void drawScaleBar(float x, float y) {
353     // Formula estandar Mercator: mpp = 40075016.686 * cos(lat) / (256 * 2^z)
354     float mpp = 40075016.686 * cos(radians(mapCenterLat))
355           / (256.0 * pow(2, mapZoom));
356
357     // Lista de distancias, se elige la que mejor se aproxima
358     // a ~150 px de longitud en pantalla
359     float[] niceDistances = {
360         10, 20, 50, 100, 200, 500, 1000, 2000, 5000, 10000
361     };
362
363     float targetPx = 150;
364     float bestDist = niceDistances[0];
365     float bestPx = bestDist / mpp;
366
367     for (float d : niceDistances) {
368         float px = d / mpp;
369         if (abs(px - targetPx) < abs(bestPx - targetPx)) {
370             bestDist = d;
371             bestPx = px;
372         }
373     }
374
375     float barLen = bestDist / mpp;
376
377     // Fondo de la caja
378     fill(MAP_PANEL_BG);
```

```

379   noStroke();
380   rect(x - 8, y - 22, barLen + 16, 32);
381
382   // Línea con topes a los extremos
383   stroke(30);
384   strokeWeight(2);
385   line(x, y, x + barLen, y);
386   line(x, y - 5, x, y + 5);
387   line(x + barLen, y - 5, x + barLen, y + 5);
388
389   fill(MAP_PANEL_TEXT);
390   noStroke();
391   textSize(11);
392   textAlign(CENTER, BOTTOM);
393   String label;
394   if (bestDist >= 1000) {
395       label = nf(bestDist / 1000, 0, 1) + " km";
396   } else {
397       label = int(bestDist) + " m";
398   }
399   text(label, x + barLen / 2, y - 6);
400 }
401
402 // Indicador de zoom + recordatorio de teclas, arriba a la izquierda
403 void drawZoomIndicator(float x, float y) {
404     fill(MAP_PANEL_BG);
405     noStroke();
406     rect(x - 5, y - 3, 235, 36);
407
408     fill(MAP_PANEL_TEXT);
409     textSize(12);
410     textAlign(LEFT, TOP);
411     text("ZOOM: " + mapZoom + " (" + MAP_ZOOM_MIN + "-" + MAP_ZOOM_MAX + ")",
412         x, y);
413     text("[+/-] Zoom    [C] Centrar    [M] PFD", x, y + 16);
414 }
415
416 // Atribucion
417 void drawAttribution(float mapW, float mapH) {
418     textSize(10);
419     textAlign(RIGHT, BOTTOM);
420     String attrib = "\u00A9 ICGC  \u00A9 OpenStreetMap contributors";
421     float tw = textWidth(attrib);
422

```

```

423 fill(MAP_PANEL_BG);
424 noStroke();
425 rect(mapW - tw - 16, mapH - 20, tw + 12, 16);
426
427 fill(MAP_PANEL_TEXT);
428 text(attrib, mapW - 10, mapH - 6);
429 }
430
431 // Overlay parpadeante cuando no hay fix
432 void drawNoFixWarning(float cx, float cy) {
433   if ((millis() / 500) % 2 == 0) { // 1 Hz
434     fill(0, 150);
435     noStroke();
436     rectMode(CENTER);
437     rect(cx, cy + 10, 320, 80);
438     rectMode(CORNER);
439
440     fill(MAP_WARNING);
441     textSize(24);
442     textAlign(CENTER, CENTER);
443     text("NO GPS FIX", cx, cy);
444     textSize(14);
445     fill(255);
446     text("Esperando satelites...", cx, cy + 30);
447   }
448 }
449
450 // Franja inferior con datos
451 void drawMapStatusStrip(float x, float y, float w) {
452   float stripH = 60;
453
454   fill(MAP_STRIP_BG);
455   noStroke();
456   rect(x, y, w, stripH);
457
458   textSize(13);
459
460   float row1Y = y + 18;
461   textAlign(LEFT, CENTER);
462
463   fill(MAP_STRIP_LABEL);
464   text("LAT", x + 15, row1Y);
465   fill(MAP_STRIP_VALUE);
466   text(nf(abs(gpsLat), 0, 5) + "\u00B0" + (gpsLat >= 0 ? "N" : "S"),

```

```

467     x + 45, row1Y);
468
469     fill(MAP_STRIP_LABEL);
470     text("LON", x + 200, row1Y);
471     fill(MAP_STRIP_VALUE);
472     text(nf(abs(gpsLon), 0, 5) + "\u00B0" + (gpsLon >= 0 ? "E" : "W"),
473         x + 235, row1Y);
474
475     fill(MAP_STRIP_LABEL);
476     text("SATS", x + 430, row1Y);
477     fill(gpsSats >= 4 ? MAP_STRIP_VALUE : MAP_WARNING); // <4 no hay solucion 3D
478     text(str(gpsSats), x + 475, row1Y);
479
480     // Semaforo de fix a la derecha
481     fill(gpsFix ? color(50, 220, 50) : color(220, 50, 50));
482     ellipse(x + w - 25, row1Y, 10, 10);
483     fill(MAP_STRIP_LABEL);
484     textAlign(RIGHT, CENTER);
485     text(gpsFix ? "FIX" : "---", x + w - 35, row1Y);
486
487     float row2Y = y + 42;
488     textAlign(LEFT, CENTER);
489
490     fill(MAP_STRIP_LABEL);
491     text("HDG", x + 15, row2Y);
492     fill(MAP_STRIP_VALUE);
493     text(nf(int(heading), 3) + "\u00B0", x + 50, row2Y);
494
495     fill(MAP_STRIP_LABEL);
496     text("GS", x + 130, row2Y);
497     fill(MAP_STRIP_VALUE);
498     text(nf(gpsSpeed, 0, 1) + " km/h", x + 155, row2Y);
499
500     fill(MAP_STRIP_LABEL);
501     text("ALT", x + 290, row2Y);
502     fill(MAP_STRIP_VALUE);
503     text(int(altitude) + " ft", x + 325, row2Y);
504
505     fill(MAP_STRIP_LABEL);
506     text("TRK", x + 430, row2Y);
507     fill(MAP_STRIP_VALUE);
508     float track = getTrackAngle();
509     text(nf(int(track), 3) + "\u00B0", x + 465, row2Y);
510

```



```

511 // Contador de puntos de traza a la derecha
512 fill(150);
513 textAlign(RIGHT, CENTER);
514 text(gpsTrail.size() + " pts", x + w - 15, row2Y);
515 }
516
517 // Angulo de track calculado entre los dos ultimos puntos de la traza
518 float getTrackAngle() {
519 // Localiza los dos ultimos puntos no nulos consecutivos
520 float[] p1 = null;
521 float[] p0 = null;
522 for (int i = gpsTrail.size() - 1; i >= 0; i--) {
523 float[] pt = gpsTrail.get(i);
524 if (pt == null) {
525 // Si hay un hueco entre los dos ultimos, el track no es calculable
526 if (p1 != null) return heading;
527 continue;
528 }
529 if (p1 == null) p1 = pt;
530 else { p0 = pt; break; }
531 }
532 if (p0 == null || p1 == null) return heading;
533
534 // Correccion de la longitud por latitud: a alta latitud un grado de
535 // longitud equivale a menos metros que un grado de latitud.
536 float dLon = (p1[1] - p0[1]) * cos(radians(p1[0]));
537 float dLat = p1[0] - p0[0];
538
539 float track = degrees(atan2(dLon, dLat));
540 if (track < 0) track += 360;
541 return track;
542 }
543
544 // Llamado desde keyPressed() en FlightDisplay.pde cuando showMap == true
545 void handleMapKeys() {
546 if (key == '+' || key == '=') {
547 mapZoom = min(MAP_ZOOM_MAX, mapZoom + 1);
548 tileCache.clear(); // un cambio de zoom invalida los tiles cargados
549 }
550 if (key == '-' || key == '_') {
551 mapZoom = max(MAP_ZOOM_MIN, mapZoom - 1);
552 tileCache.clear();
553 }
554 if (key == 'c' || key == 'C') {

```

```
555     mapCenterLocked = true;
556   }
557   if (key == 'x' || key == 'X') {
558     gpsTrail.clear();
559     println("Traza GPS borrada");
560   }
561 }
```

Código 2.4: Código de la representación del mapa y datos GPS (GPS Map)

```
1  // Indicador de ground speed (GS, knots) desde GPS
2  // Cinta vertical centrada en la velocidad actual. Marcas cada 5 kt,
3  // numeradas cada 10
4
5  void drawGroundSpeedIndicator(float centerX, float centerY, float tapeWidth,
6    float tapeHeight) {
7    float halfH = tapeHeight / 2;
8    float halfW = tapeWidth / 2;
9    float rangeVisible = 80;           // knots visibles en la cinta
10   float pxPerKt = tapeHeight / rangeVisible;
11
12   // Fondo
13   fill(40);
14   stroke(80);
15   strokeWeight(2);
16   rectMode(CENTER);
17   rect(centerX, centerY, tapeWidth, tapeHeight);
18
19   // Marcas
20   float minSpd = groundSpeed - (rangeVisible / 2) - 10;
21   float maxSpd = groundSpeed + (rangeVisible / 2) + 10;
22   int startTick = ceil(minSpd / 5.0) * 5;
23
24   for (int spdVal = startTick; spdVal <= maxSpd; spdVal += 5) {
25     if (spdVal < 0) continue;         // no se dibujan velocidades negativas
26
27     float delta = spdVal - groundSpeed;
28     float y = centerY - (delta * pxPerKt);
29
30     if (y < centerY - halfH + 2 || y > centerY + halfH - 2) continue;
31
32     float xRight = centerX + halfW;
```

```
32     boolean isMajor = (spdVal % 10 == 0);
33     float tickLen = isMajor ? 12 : 6;
34     float weight = isMajor ? 2 : 1;
35
36     stroke(255);
37     strokeWeight(weight);
38     line(xRight, y, xRight - tickLen, y);
39
40     if (isMajor && spdVal > 0) {
41         fill(255);
42         textAlign(RIGHT, CENTER);
43         textSize(15);
44         text(str(spdVal), xRight - tickLen - 4, y - 1);
45     }
46 }
47
48 // Readout central
49 // El triangulo en el lado derecho de la caja indica el valor actual
50 float boxW = 55;
51 float boxH = 16;
52 float boxX = centerX + halfW - 5;
53
54 fill(0);
55 stroke(255);
56 strokeWeight(2);
57 beginShape();
58 vertex(boxX - boxW, centerY - boxH);
59 vertex(boxX, centerY - boxH);
60 vertex(boxX, centerY - 6);
61 vertex(boxX + 6, centerY);
62 vertex(boxX, centerY + 6);
63 vertex(boxX, centerY + boxH);
64 vertex(boxX - boxW, centerY + boxH);
65 endShape(CLOSE);
66
67 fill(0, 255, 0);
68 textAlign(CENTER, CENTER);
69 textSize(18);
70 text(int(groundSpeed), boxX - boxW / 2, centerY - 1);
71
72 // Etiqueta arriba ("GS") y abajo ("KTS")
73 fill(180);
74 textAlign(CENTER, TOP);
75 textSize(11);
```

```

76  text("GS", centerX, centerY - halfH + 4);
77
78  fill(180);
79  textAlign(CENTER, BOTTOM);
80  textSize(11);
81  text("KTS", centerX, centerY + halfH - 4);
82
83  rectMode(CORNER);
84  textAlign(LEFT);
85  }

```

Código 2.5: Código del indicador de velocidad respecto al suelo (Ground Speed Indicator)

```

1  // Indicador de rumbo con heading bug
2  // Cinta horizontal centrada en el heading actual, con etiquetas cardinales
3  // (N/E/S/W), marcas cada 5deg, 10deg y 30deg, y un bug cyan ajustable por el
4  // piloto con las flechas izquierda/derecha
5
6  void drawHeadingIndicator(float centerX, float centerY, float tapeWidth, float
    tapeHeight) {
7      float halfW = tapeWidth / 2;
8      float degreesVisible = 60;           // rango total visible en la cinta
9      float pxPerDeg = tapeWidth / degreesVisible;
10
11     // Fondo
12     fill(40);
13     stroke(80);
14     strokeWeight(2);
15     rectMode(CENTER);
16     rect(centerX, centerY, tapeWidth, tapeHeight);
17
18     // Heading bug
19     // Diferencia circular entre bug y heading
20     float bugDelta = targetHdgBug - heading;
21     if (bugDelta > 180) bugDelta -= 360;
22     if (bugDelta < -180) bugDelta += 360;
23     float bugX = centerX + (bugDelta * pxPerDeg);
24
25     // El bug solo se dibuja si cae dentro de la cinta visible
26     if (bugX > centerX - halfW + 8 && bugX < centerX + halfW - 8) {
27         fill(0, 255, 255);
28         noStroke();

```

```
29     float bugTop = centerY - tapeHeight / 2;
30
31     // Forma del bug: una pequena "U" invertida
32     beginShape();
33     vertex(bugX - 8, bugTop + 1);
34     vertex(bugX - 8, bugTop + 13);
35     vertex(bugX - 4, bugTop + 13);
36     vertex(bugX - 4, bugTop + 5);
37     vertex(bugX + 4, bugTop + 5);
38     vertex(bugX + 4, bugTop + 13);
39     vertex(bugX + 8, bugTop + 13);
40     vertex(bugX + 8, bugTop + 1);
41     endShape(CLOSE);
42 }
43
44 // Marcas
45 float minDeg = heading - (degreesVisible / 2) - 5;
46 float maxDeg = heading + (degreesVisible / 2) + 5;
47 int startTick = ceil(minDeg / 5.0) * 5;
48
49 for (int tickVal = startTick; tickVal <= maxDeg; tickVal += 5) {
50     float delta = tickVal - heading;
51     float x = centerX + (delta * pxPerDeg);
52
53     if (x < centerX - halfW + 2 || x > centerX + halfW - 2) continue;
54
55     // Normaliza el valor a [0, 360)
56     int displayDeg = tickVal % 360;
57     if (displayDeg < 0) displayDeg += 360;
58
59     float tickLen;
60     float weight;
61     boolean showText = false;
62     int textSz = 16;
63
64     // Marcas mayores cada 30deg: cardinales o numero grande
65     if (displayDeg % 30 == 0) {
66         tickLen = 25;
67         weight = 3;
68         showText = true;
69         textSz = 20;
70     } else if (displayDeg % 10 == 0) {
71         tickLen = 15;
72         weight = 2;
```

```
73     showText = true;
74     textSz = 16;
75 } else {
76     tickLen = 8;
77     weight = 1;
78 }
79
80 stroke(255);
81 strokeWeight(weight);
82 float tickTop = centerY - tapeHeight / 2;
83 line(x, tickTop, x, tickTop + tickLen);
84
85 if (showText) {
86     fill(255);
87     textAlign(CENTER, TOP);
88     textSize(textSz);
89
90     // Cardinales en las cuatro direcciones principales. El resto de
91     // marcas mayores se etiquetan como numero/10 segun la convencion
92     // aeronautica (090 deg se muestra como "9").
93     String label;
94     if (displayDeg == 0)         label = "N";
95     else if (displayDeg == 90)  label = "E";
96     else if (displayDeg == 180) label = "S";
97     else if (displayDeg == 270) label = "W";
98     else label = str(displayDeg / 10);
99
100     text(label, x, tickTop + tickLen + 3);
101 }
102 }
103
104 // Puntero fijo en el centro (triangulo amarillo apuntando hacia abajo)
105 fill(255, 200, 0);
106 noStroke();
107 triangle(centerX - 10, centerY - tapeHeight / 2 - 2,
108         centerX + 10, centerY - tapeHeight / 2 - 2,
109         centerX, centerY - tapeHeight / 2 + 15);
110
111 // Readout numerico del heading actual
112 float boxY = centerY + tapeHeight / 2 + 20;
113 fill(0);
114 stroke(255);
115 strokeWeight(2);
116 rectMode(CENTER);
```

```

117  rect(centerX, boxY, 64, 26);
118
119  fill(0, 255, 0);
120  textAlign(CENTER, CENTER);
121  textSize(18);
122  float hdgDisplay = heading % 360;
123  if (hdgDisplay < 0) hdgDisplay += 360;
124  text(nf(int(hdgDisplay), 3) + "\u00B0", centerX, boxY - 2);
125
126  // Valor del bug al lado
127  fill(0, 255, 255);
128  textAlign(LEFT, CENTER);
129  textSize(12);
130  text("HDG " + nf(int(targetHdgBug), 3), centerX + 40, boxY - 2);
131
132  rectMode(CORNER);
133  textAlign(LEFT);
134 }

```

Código 2.6: Código del indicador de rumbo (Heading Indicator)

```

1  // Indicador de proximidad al suelo
2  // Muestra la distancia mas cercana medida por los sensores y un codigo
3  // de color por umbrales. La medida del radar HLK-LD2410C se recibe pero
4  // queda fuera de la decision por falta de calidad en banco, el ToF
5  // VL53L1X es la unica fuente
6
7  void drawProximityIndicator(float centerX, float centerY, float boxWidth, float
    boxHeight) {
8      // Conversion a cm para tratar ambas medidas en la misma unidad
9      float tofDistCm = tofDist / 10.0;
10     float radarDistCm = radarDist;
11
12     // Seleccion de la distancia activa
13     // El radar queda excluido: sus medidas son poco fiables actualmente
14     // La variable se conserva por si en un futuro se reactiva su contribucion
15     float currentDist = Float.MAX_VALUE;
16     String activeSensor = "NONE";
17
18     if (tofDistCm > 0 && tofDistCm < currentDist) {
19         currentDist = tofDistCm;
20         activeSensor = "LIDAR";

```

```
21 }
22 // Rama del radar deshabilitada:
23 // if (radarDistCm > 0 ES radarDistCm < currentDist) {
24 //   currentDist = radarDistCm;
25 //   activeSensor = "RADAR";
26 // }
27
28 // Caja de fondo con esquinas redondeadas
29 fill(40);
30 stroke(80);
31 strokeWeight(2);
32 rectMode(CENTER);
33 rect(centerX, centerY, boxWidth, boxHeight, 8);
34
35 // Titulo
36 fill(180);
37 textAlign(CENTER, TOP);
38 textSize(12);
39 text("PROXIMITY", centerX, centerY - boxHeight / 2 + 4);
40
41 // Estados de salida: sin datos, fuera de rango o medida activa
42 float safeDistanceThreshold = 500;
43
44 if (currentDist == Float.MAX_VALUE) {
45   // No hay lectura valida
46   fill(130);
47   textAlign(CENTER, CENTER);
48   textSize(18);
49   text("NO DATA", centerX, centerY);
50
51 } else if (currentDist > safeDistanceThreshold) {
52   // Distancia por encima del umbral de seguridad
53   fill(100, 255, 100);
54   textAlign(CENTER, CENTER);
55   textSize(22);
56   text("SAFE", centerX, centerY);
57
58   fill(130);
59   textSize(10);
60   text("OUT OF RANGE", centerX, centerY + 18);
61
62 } else {
63   // Medida valida en zona de aterrizaje: color segun umbral
64   color alertColor;
```



```

65     if (currentDist <= 50) {
66         alertColor = color(255, 50, 50);    // rojo: < 50 cm
67     } else if (currentDist <= 150) {
68         alertColor = color(255, 200, 0);    // amarillo: 50 - 150 cm
69     } else {
70         alertColor = color(50, 255, 50);    // verde: > 150 cm
71     }
72
73     fill(alertColor);
74     textAlign(CENTER, CENTER);
75     textSize(26);
76     text(int(currentDist) + " cm", centerX, centerY - 2);
77
78     // Etiqueta del sensor activo
79     fill(180);
80     textSize(11);
81     text(activeSensor, centerX, centerY + 20);
82 }
83
84 // Restaurar modos de dibujo por defecto
85 rectMode(CORNER);
86 textAlign(LEFT);
87 }

```

Código 2.7: Código del indicador de proximidad (Proximity Indicator)

```

1  // Indicador de velocidad vertical (VSI)
2  // Fondo de escala reducido respecto al estandar aeronautico para adaptar
3  // el instrumento a la operacion en banco de pruebas
4
5  void drawVSI(float centerX, float centerY, float tapeWidth, float tapeHeight) {
6      float halfH = tapeHeight / 2;
7      float halfW = tapeWidth / 2;
8
9      // Fondo de escala
10     float maxVS = 100;
11     float padding = 10; // Añade 10 píxeles de margen arriba y abajo
12     float pxPerFPM = (halfH - padding) / maxVS;
13
14
15     // Fondo
16     fill(40);

```

```
17  stroke(80);
18  strokeWeight(2);
19  rectMode(CENTER);
20  rect(centerX, centerY, tapeWidth, tapeHeight);
21
22  // Línea del cero
23  stroke(255);
24  strokeWeight(1.5);
25  line(centerX - halfW, centerY, centerX + halfW, centerY);
26
27  // Marcas cada 25 fpm, mayores cada 50
28  for (int vs = -100; vs <= 100; vs += 25) {
29      float y = centerY - (vs * pxPerFPM);
30      float xLeft = centerX - halfW;
31
32      boolean isMajor = (vs % 50 == 0);
33      float tickLen = isMajor ? 10 : 6;
34      float weight = isMajor ? 1.5 : 1;
35
36      stroke(255);
37      strokeWeight(weight);
38      line(xLeft, y, xLeft + tickLen, y);
39
40      // Solo se etiquetan las marcas mayores, excluyendo el cero
41      if (isMajor && vs != 0) {
42          fill(255);
43          textAlign(CENTER, CENTER);
44          textSize(11);
45          text(str(abs(vs)), centerX + 12, y);
46      }
47  }
48
49  // Barra de tendencia (rellena desde 0 hasta el valor actual)
50  float clampedVS = constrain(verticalSpeed, -maxVS, maxVS);
51  float vsY = centerY - (clampedVS * pxPerFPM);
52
53  // Color de la barra en funcion de la magnitud (verde/amarillo/naranja)
54  float absVS = abs(clampedVS);
55  int barR, barG, barB;
56  if (absVS < 50) {
57      barR = 0;   barG = 200; barB = 0;
58  } else if (absVS < 75) {
59      barR = 180; barG = 200; barB = 0;
60  } else {
```

```
61     barR = 255; barG = 180; barB = 0;
62 }
63
64 float barX = centerX - halfW + 2;
65 float barW = tapeWidth - 16;
66
67 noStroke();
68 fill(barR, barG, barB, 130);
69
70 rectMode(CORNERS);
71 if (clampedVS > 0) {
72     rect(barX, vsY, barX + barW, centerY);
73 } else if (clampedVS < 0) {
74     rect(barX, centerY, barX + barW, vsY);
75 }
76 rectMode(CORNER);
77
78 // Puntero (triangulo verde apuntando a la cinta desde la izquierda)
79 fill(0, 255, 0);
80 noStroke();
81 float triX = centerX - halfW;
82 triangle(triX, vsY - 4, triX, vsY + 4, triX + 8, vsY);
83
84 // Readout numerico arriba del instrumento
85 float rdY = centerY - halfH - 14;
86 fill(0);
87 stroke(80);
88 strokeWeight(1);
89 rectMode(CENTER);
90 rect(centerX, rdY, tapeWidth + 2, 20);
91
92 fill(0, 255, 0);
93 textAlign(CENTER, CENTER);
94 textSize(12);
95
96 // Valor sin redondear a la decena, para ver variaciones pequenas
97 int displayVS = round(verticalSpeed);
98 String vsText;
99 if (displayVS > 0) vsText = "+" + displayVS;
100 else vsText = str(displayVS);
101 text(vsText, centerX, rdY - 1);
102
103 // Etiqueta "VS"
104 fill(150);
```

```
105   textAlign(CENTER, BOTTOM);
106   textSize(10);
107   text("VS", centerX, rdY - 12);
108
109   rectMode(CORNER);
110   textAlign(LEFT);
111 }
```

Código 2.8: Código del indicador de velocidad vertical (VSI)

Capítulo 3

ANEXO III. Figuras de apoyo

A continuación de muestras las fotos de la comprobación de la actitud de la plataforma.















